

ARTIFICIAL
INTELLIGENCE

THIRD EDITION

IAN MILLINGTON



CRC Press
Taylor & Francis Group

AI for Games

THIRD EDITION



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

AI for **Games**

THIRD EDITION

Ian Millington



CRC Press

Taylor & Francis Group

Boca Raton London New York

CRC Press is an imprint of the
Taylor & Francis Group, an **informa** business

CRC Press
Taylor & Francis Group
6000 Broken Sound Parkway NW, Suite 300
Boca Raton, FL 33487-2742

© 2019 by Taylor & Francis Group, LLC
CRC Press is an imprint of Taylor & Francis Group, an Informa business

No claim to original U.S. Government works

Printed on acid-free paper
Version Date: 20190221

International Standard Book Number-13: 978-1-138-48397-2 (Hardback)

This book contains information obtained from authentic and highly regarded sources. Reasonable efforts have been made to publish reliable data and information, but the author and publisher cannot assume responsibility for the validity of all materials or the consequences of their use. The authors and publishers have attempted to trace the copyright holders of all material reproduced in this publication and apologize to copyright holders if permission to publish in this form has not been obtained. If any copyright material has not been acknowledged please write and let us know so we may rectify in any future reprint.

Except as permitted under U.S. Copyright Law, no part of this book may be reprinted, reproduced, transmitted, or utilized in any form by any electronic, mechanical, or other means, now known or hereafter invented, including photocopying, microfilming, and recording, or in any information storage or retrieval system, without written permission from the publishers.

For permission to photocopy or use material electronically from this work, please access www.copyright.com (<http://www.copyright.com/>) or contact the Copyright Clearance Center, Inc. (CCC), 222 Rosewood Drive, Danvers, MA 01923, 978-750-8400. CCC is a not-for-profit organization that provides licenses and registration for a variety of users. For organizations that have been granted a photocopy license by the CCC, a separate system of payment has been arranged.

Trademark Notice: Product or corporate names may be trademarks or registered trademarks, and are used only for identification and explanation without intent to infringe.

Library of Congress Cataloging-in-Publication Data

Names: Millington, Ian, author.
Title: AI for games / Ian Millington.
Other titles: AI for games
Description: Third edition. | Boca Raton : Taylor & Francis, a CRC title, part of the Taylor & Francis imprint, a member of the Taylor & Francis Group, the academic division of T&F Informa, plc, [2019] | Includes bibliographical references.
Identifiers: LCCN 2018041305 | ISBN 9781138483972 (hardback : acid-free paper)
Subjects: LCSH: Computer games--Programming. | Computer animation. | Artificial intelligence.
Classification: LCC QA76.76.C672 M549 2019 | DDC 794.8/1525--dc23
LC record available at <https://lcn.loc.gov/2018041305>

Visit the Taylor & Francis Web site at
<http://www.taylorandfrancis.com>
and the CRC Press Web site at
<http://www.crcpress.com>

for Daniel
my favorite game designer



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

CONTENTS

PART I

AI and Games

1

CHAPTER 1 INTRODUCTION

3

1

1.1	What Is AI?	4
1.1.1	Academic AI	5
1.1.2	Game AI	8
1.2	Model of Game AI	10
1.2.1	Movement	11
1.2.2	Decision Making	11
1.2.3	Strategy	12
1.2.4	Infrastructure	12
1.2.5	Agent-Based AI	13
1.2.6	In the Book	13
1.3	Algorithms and Data Structures	14
1.3.1	Algorithms	14
1.3.2	Representations	17
1.3.3	Implementation	18
1.4	Layout of the Book	18

CHAPTER 2 GAME AI

21

2

2.1	The Complexity Fallacy	21
2.1.1	When Simple Things Look Good	21
2.1.2	When Complex Things Look Bad	22
2.1.3	The Perception Window	23
2.1.4	Changes of Behavior	24
2.2	The Kind of AI in Games	24
2.2.1	Hacks	25
2.2.2	Heuristics	26
2.2.3	Algorithms	27

2.3	Speed and Memory Constraints	28
2.3.1	Processor Issues	28
2.3.2	Memory Concerns	30
2.3.3	Platforms	32
2.4	The AI Engine	35
2.4.1	Structure of an AI Engine	35
2.4.2	Tool Concerns	37
2.4.3	Putting It All Together	37

PART II

Techniques

39

CHAPTER 3 MOVEMENT

41

3

3.1	The Basics of Movement Algorithms	42
3.1.1	Two-Dimensional Movement	43
3.1.2	Statics	44
3.1.3	Kinematics	47
3.2	Kinematic Movement Algorithms	51
3.2.1	Seek	51
3.2.2	Wandering	54
3.3	Steering Behaviors	56
3.3.1	Steering Basics	56
3.3.2	Variable Matching	57
3.3.3	Seek and Flee	58
3.3.4	Arrive	60
3.3.5	Align	63
3.3.6	Velocity Matching	67
3.3.7	Delegated Behaviors	68
3.3.8	Pursue and Evade	68
3.3.9	Face	72
3.3.10	Looking Where You're Going	72
3.3.11	Wander	73
3.3.12	Path Following	76
3.3.13	Separation	82
3.3.14	Collision Avoidance	85
3.3.15	Obstacle and Wall Avoidance	90
3.3.16	Summary	95

3.4	Combining Steering Behaviors	95
3.4.1	Blending and Arbitration	96
3.4.2	Weighted Blending	96
3.4.3	Priorities	103
3.4.4	Cooperative Arbitration	106
3.4.5	Steering Pipeline	108
3.5	Predicting Physics	120
3.5.1	Aiming and Shooting	120
3.5.2	Projectile Trajectory	121
3.5.3	The Firing Solution	123
3.5.4	Projectiles with Drag	126
3.5.5	Iterative Targeting	128
3.6	Jumping	133
3.6.1	Jump Points	134
3.6.2	Landing Pads	138
3.6.3	Hole Fillers	142
3.7	Coordinated Movement	143
3.7.1	Fixed Formations	144
3.7.2	Scalable Formations	145
3.7.3	Emergent Formations	146
3.7.4	Two-Level Formation Steering	147
3.7.5	Implementation	151
3.7.6	Extending to More Than Two Levels	156
3.7.7	Slot Roles and Better Assignment	158
3.7.8	Slot Assignment	160
3.7.9	Dynamic Slots and Plays	164
3.7.10	Tactical Movement	166
3.8	Motor Control	170
3.8.1	Output Filtering	171
3.8.2	Capability-Sensitive Steering	173
3.8.3	Common Actuation Properties	174
3.9	Movement in the Third Dimension	177
3.9.1	Rotation in Three Dimensions	178
3.9.2	Converting Steering Behaviors to Three Dimensions	179
3.9.3	Align	180
3.9.4	Align to Vector	181
3.9.5	Face	182
3.9.6	Look Where You're Going	185
3.9.7	Wander	185
3.9.8	Faking Rotation Axes	187

CHAPTER	PATHFINDING	195
4		
4.1	The Pathfinding Graph	196
4.1.1	Graphs	197
4.1.2	Weighted Graphs	198
4.1.3	Directed Weighted Graphs	200
4.1.4	Terminology	202
4.1.5	Representation	202
4.2	Dijkstra	203
4.2.1	The Problem	203
4.2.2	The Algorithm	204
4.2.3	Pseudo-Code	209
4.2.4	Data Structures and Interfaces	211
4.2.5	Performance of Dijkstra	213
4.2.6	Weaknesses	213
4.3	A*	215
4.3.1	The Problem	215
4.3.2	The Algorithm	215
4.3.3	Pseudo-Code	219
4.3.4	Data Structures and Interfaces	222
4.3.5	Implementation Notes	227
4.3.6	Algorithm Performance	227
4.3.7	Node Array A*	228
4.3.8	Choosing a Heuristic	230
4.4	World Representations	237
4.4.1	Tile Graphs	239
4.4.2	Dirichlet Domains	242
4.4.3	Points of Visibility	244
4.4.4	Navigation Meshes	247
4.4.5	Non-Translational Problems	252
4.4.6	Cost Functions	252
4.4.7	Path Smoothing	253
4.5	Improving on A*	257
4.6	Hierarchical Pathfinding	257
4.6.1	The Hierarchical Pathfinding Graph	258
4.6.2	Pathfinding on the Hierarchical Graph	261
4.6.3	Hierarchical Pathfinding on Exclusions	264
4.6.4	Strange Effects of Hierarchies on Pathfinding	265
4.6.5	Instanced Geometry	267
4.7	Other Ideas in Pathfinding	274
4.7.1	Open Goal Pathfinding	274

4.7.2	Dynamic Pathfinding	274
4.7.3	Other Kinds of Information Reuse	275
4.7.4	Low Memory Algorithms	275
4.7.5	Interruptible Pathfinding	277
4.7.6	Pooling Planners	277
4.8	Continuous Time Pathfinding	278
4.8.1	The Problem	279
4.8.2	The Algorithm	280
4.8.3	Implementation Notes	284
4.8.4	Performance	284
4.8.5	Weaknesses	285
4.9	Movement Planning	285
4.9.1	Animations	285
4.9.2	Movement Planning	287
4.9.3	Example	289
4.9.4	Footfalls	291
CHAPTER 5	DECISION MAKING	297
5.1	Overview of Decision Making	298
5.2	Decision Trees	299
5.2.1	The Problem	300
5.2.2	The Algorithm	300
5.2.3	Pseudo-Code	305
5.2.4	Knowledge Representation	307
5.2.5	Implementation Notes	308
5.2.6	Performance of Decision Trees	308
5.2.7	Balancing the Tree	309
5.2.8	Beyond the Tree	310
5.2.9	Random Decision Trees	311
5.3	State Machines	314
5.3.1	The Problem	316
5.3.2	The Algorithm	316
5.3.3	Pseudo-Code	316
5.3.4	Data Structures and Interfaces	317
5.3.5	Performance	320
5.3.6	Implementation Notes	320
5.3.7	Hard-Coded FSM	320
5.3.8	Hierarchical State Machines	323
5.3.9	Combining Decision Trees and State Machines	335

5.4 Behavior Trees	338
5.4.1 Implementing Behavior Trees	345
5.4.2 Pseudo-Code	345
5.4.3 Decorators	349
5.4.4 Concurrency and Timing	355
5.4.5 Adding Data to Behavior Trees	364
5.4.6 Reusing Trees	369
5.4.7 Limitations of Behavior Trees	373
5.5 Fuzzy Logic	374
5.5.1 A Warning	375
5.5.2 Introduction to Fuzzy Logic	375
5.5.3 Fuzzy Logic Decision Making	385
5.5.4 Fuzzy State Machines	394
5.6 Markov Systems	399
5.6.1 Markov Processes	400
5.6.2 Markov State Machine	402
5.7 Goal-Oriented Behavior	405
5.7.1 Goal-Oriented Behavior	406
5.7.2 Simple Selection	408
5.7.3 Overall Utility	410
5.7.4 Timing	412
5.7.5 Overall Utility GOAP	416
5.7.6 GOAP with IDA*	422
5.7.7 Smelly GOB	429
5.8 Rule-Based Systems	431
5.8.1 The Problem	431
5.8.2 The Algorithm	436
5.8.3 Pseudo-Code	437
5.8.4 Data Structures and Interfaces	438
5.8.5 Rule Arbitration	444
5.8.6 Unification	446
5.8.7 Rete	448
5.8.8 Extensions	457
5.8.9 Where Next	461
5.9 Blackboard Architectures	461
5.9.1 The Problem	462
5.9.2 The Algorithm	462
5.9.3 Pseudo-Code	464
5.9.4 Data Structures and Interfaces	464
5.9.5 Performance	467
5.9.6 Other Things Are Blackboard Systems	467

5.10	Action Execution	469
5.10.1	Types of Action	469
5.10.2	The Algorithm	474
5.10.3	Pseudo-Code	475
5.10.4	Data Structures and Interfaces	476
5.10.5	Implementation Notes	478
5.10.6	Performance	479
5.10.7	Putting It All Together	479
CHAPTER 6	TACTICAL AND STRATEGIC AI	485
6.1	Waypoint Tactics	486
6.1.1	Tactical Locations	486
6.1.2	Using Tactical Locations	496
6.1.3	Generating the Tactical Properties of a Waypoint	500
6.1.4	Automatically Generating the Waypoints	506
6.1.5	The Condensation Algorithm	507
6.2	Tactical Analyses	511
6.2.1	Representing the Game Level	512
6.2.2	Simple Influence Maps	512
6.2.3	Terrain Analysis	519
6.2.4	Learning with Tactical Analyses	521
6.2.5	A Structure for Tactical Analyses	523
6.2.6	Map Flooding	528
6.2.7	Convolution Filters	532
6.2.8	Cellular Automata	542
6.3	Tactical Pathfinding	548
6.3.1	The Cost Function	548
6.3.2	Tactic Weights and Concern Blending	549
6.3.3	Modifying the Pathfinding Heuristic	551
6.3.4	Tactical Graphs for Pathfinding	552
6.3.5	Using Tactical Waypoints	553
6.4	Coordinated Action	554
6.4.1	Multi-Tier AI	554
6.4.2	Emergent Cooperation	561
6.4.3	Scripting Group Actions	564
6.4.4	Military Tactics	568
CHAPTER 7	LEARNING	575
7.1	Learning Basics	576
7.1.1	Online or Offline Learning	576

7.1.2	Intra-Behavior Learning	577
7.1.3	Inter-Behavior Learning	577
7.1.4	A Warning	577
7.1.5	Over-Learning	578
7.1.6	The Zoo of Learning Algorithms	579
7.1.7	The Balance of Effort	579
7.2	Parameter Modification	579
7.2.1	The Parameter Landscape	580
7.2.2	Hill Climbing	581
7.2.3	Extensions to Basic Hill Climbing	584
7.2.4	Annealing	588
7.3	Action Prediction	592
7.3.1	Left or Right	592
7.3.2	Raw Probability	593
7.3.3	String Matching	593
7.3.4	N-Grams	593
7.3.5	Window Size	597
7.3.6	Hierarchical N-Grams	599
7.3.7	Application in Combat	602
7.4	Decision Learning	602
7.4.1	The Structure of Decision Learning	602
7.4.2	What Should You Learn?	603
7.4.3	Four Techniques	604
7.5	Naïve Bayes Classifiers	604
7.5.1	Pseudo-Code	607
7.5.2	Implementation Notes	609
7.6	Decision Tree Learning	609
7.6.1	ID3	610
7.6.2	ID3 with Continuous Attributes	618
7.6.3	Incremental Decision Tree Learning	622
7.7	Reinforcement Learning	627
7.7.1	The Problem	627
7.7.2	The Algorithm	628
7.7.3	Pseudo-Code	631
7.7.4	Data Structures and Interfaces	632
7.7.5	Implementation Notes	633
7.7.6	Performance	633
7.7.7	Tailoring Parameters	633
7.7.8	Weaknesses and Realistic Applications	637
7.7.9	Other Ideas in Reinforcement Learning	640

7.8	Artificial Neural Networks	642
7.8.1	Overview	643
7.8.2	The Problem	646
7.8.3	The Algorithm	647
7.8.4	Pseudo-Code	651
7.8.5	Data Structures and Interfaces	653
7.8.6	Implementation Caveats	655
7.8.7	Performance	655
7.8.8	Other Approaches	655
7.9	Deep Learning	660
7.9.1	What is Deep Learning?	660
7.9.2	Data	661
CHAPTER	PROCEDURAL CONTENT GENERATION	669
8		
8.1	Pseudorandom Numbers	670
8.1.1	Numeric Mixing and Game Seeds	671
8.1.2	Halton Sequence	672
8.1.3	Phyllotaxic Angles	675
8.1.4	Poisson Disk	676
8.2	Lindenmayer Systems	680
8.2.1	Simple L-systems	680
8.2.2	Adding Randomness to L-systems	684
8.2.3	Stage-Specific Rules	686
8.3	Landscape Generation	687
8.3.1	Modifiers and Height-Maps	688
8.3.2	Noise	689
8.3.3	Perlin Noise	689
8.3.4	Faults	692
8.3.5	Thermal Erosion	694
8.3.6	Hydraulic Erosion	696
8.3.7	Altitude Filtering	701
8.4	Dungeons and Maze Generation	705
8.4.1	Mazes by Depth First Backtracking	705
8.4.2	Minimum Spanning Tree Algorithms	716
8.4.3	Recursive Subdivision	721
8.4.4	Generate and Test	727
8.5	Shape Grammars	727
8.5.1	Running the Grammar	730
8.5.2	Planning	733

CHAPTER	BOARD GAMES	741
9		
9.1	Game Theory	742
9.1.1	Types of Games	742
9.1.2	The Game Tree	744
9.2	Minimaxing	747
9.2.1	The Static Evaluation Function	747
9.2.2	Minimaxing	749
9.2.3	The Minimaxing Algorithm	750
9.2.4	Negamaxing	753
9.2.5	AB Pruning	755
9.2.6	The AB Search Window	759
9.2.7	Negascout	761
9.3	Transposition Tables and Memory	764
9.3.1	Hashing Game States	764
9.3.2	What to Store in the Table	767
9.3.3	Hash Table Implementation	768
9.3.4	Replacement Strategies	769
9.3.5	A Complete Transposition Table	770
9.3.6	Transposition Table Issues	771
9.3.7	Using Opponent's Thinking Time	771
9.4	Memory-Enhanced Test Algorithms	772
9.4.1	Implementing Test	772
9.4.2	The MTD Algorithm	774
9.4.3	Pseudo-Code	775
9.5	Monte Carlo Tree Search (MCTS)	776
9.5.1	Pure Monte Carlo Tree Search	776
9.5.2	Adding Knowledge	782
9.6	Opening Books and Other Set Plays	783
9.6.1	Implementing an Opening Book	783
9.6.2	Learning for Opening Books	784
9.6.3	Set Play Books	785
9.7	Further Optimizations	785
9.7.1	Iterative Deepening	786
9.7.2	Variable Depth Approaches	787
9.8	Game Knowledge	788
9.8.1	Creating a Static Evaluation Function	791
9.8.2	Learning the Static Evaluation Function	794
9.9	Turn-Based Strategy Games	797
9.9.1	Impossible Tree Size	798
9.9.2	Real-Time AI in a Turn-Based Game	799

PART III**Supporting Technologies****801**

CHAPTER	EXECUTION MANAGEMENT	803
10		
	10.1 Scheduling	804
	10.1.1 The Scheduler	804
	10.1.2 Interruptible Processes	811
	10.1.3 Load-Balancing Scheduler	814
	10.1.4 Hierarchical Scheduling	816
	10.1.5 Priority Scheduling	818
	10.2 Anytime Algorithms	820
	10.3 Level of Detail	821
	10.3.1 Graphics Level of Detail	821
	10.3.2 AI LOD	822
	10.3.3 Scheduling LOD	823
	10.3.4 Behavioral LOD	824
	10.3.5 Group LOD	829
	10.3.6 In Summary	833
CHAPTER	WORLD INTERFACING	835
11		
	11.1 Communication	836
	11.1.1 Polling	836
	11.1.2 Events	837
	11.1.3 Determining Which Approach to Use	838
	11.2 Event Managers	838
	11.2.1 Implementation	841
	11.2.2 Event Casting	843
	11.2.3 Inter-Agent Communication	846
	11.3 Polling Stations	846
	11.3.1 Pseudo-Code	847
	11.3.2 Performance	848
	11.3.3 Implementation Notes	848
	11.3.4 Abstract Polling	848
	11.4 Sense Management	849
	11.4.1 Faking It	850
	11.4.2 What Do We Know?	851
	11.4.3 Sensory Modalities	852
	11.4.4 Region Sense Manager	857
	11.4.5 Finite Element Model Sense Manager	865

CHAPTER	TOOLS AND CONTENT CREATION	875
12		
12.0.1	Toolchains Limit AI	876
12.0.2	Where AI Knowledge Comes From	876
12.1	Knowledge for Pathfinding and Waypoints	877
12.1.1	Manually Creating Region Data	877
12.1.2	Automatic Graph Creation	880
12.1.3	Geometric Analysis	880
12.1.4	Data Mining	884
12.2	Knowledge for Movement	886
12.2.1	Obstacles	886
12.2.2	High-Level Staging	888
12.3	Knowledge for Decision Making	889
12.3.1	Object Types	889
12.3.2	Concrete Actions	889
12.4	The Toolchain	890
12.4.1	Integrated Game Engines	890
12.4.2	Custom Data-Driven Editors	893
12.4.3	AI Design Tools	894
12.4.4	Remote Debugging	895
12.4.5	Plug-Ins	896
CHAPTER	PROGRAMMING GAME AI	901
13		
13.1	Implementation Language	902
13.1.1	C++	903
13.1.2	C#	904
13.1.3	Swift	905
13.1.4	Java	906
13.1.5	JavaScript	908
13.2	Scripted AI	909
13.2.1	What Is Scripted AI?	910
13.2.2	What Makes a Good Scripting Language?	911
13.2.3	Embedding	913
13.2.4	Choosing an Open Source Language	913
13.2.5	A Language Selection	914
13.3	Creating a Language	919
13.3.1	Rolling Your Own	921

PART IV**Designing Game AI****927**

CHAPTER	DESIGNING GAME AI	929
14	14.1 The Design	929
	14.1.1 Example	931
	14.1.2 Evaluating the Behaviors	931
	14.1.3 Selecting Techniques	934
	14.1.4 The Scope of One Game	936
	14.2 Shooters	937
	14.2.1 Movement and Firing	937
	14.2.2 Decision Making	939
	14.2.3 Perception	940
	14.2.4 Pathfinding and Tactical AI	941
	14.2.5 Shooter-Like Games	942
	14.2.6 Melee Combat	943
	14.3 Driving	946
	14.3.1 Movement	946
	14.3.2 Pathfinding and Tactical AI	949
	14.3.3 Driving-Like Games	950
	14.4 Real-Time Strategy	950
	14.4.1 Pathfinding	950
	14.4.2 Group Movement	951
	14.4.3 Tactical and Strategic AI	952
	14.4.4 Decision Making	953
	14.4.5 MOBAs	954
	14.5 Sports	955
	14.5.1 Physics Prediction	956
	14.5.2 Playbooks and Content Creation	956
	14.6 Turn-Based Strategy Games	956
	14.6.1 Timing	957
	14.6.2 Helping the Player	958
CHAPTER	AI-BASED GAME GENRES	959
15	15.1 Teaching Characters	960
	15.1.1 Representing Actions	960
	15.1.2 Representing the World	961
	15.1.3 Learning Mechanism	961
	15.1.4 Predictable Mental Models	963

15.2 Flocking and Herding Games	965
15.2.1 Making the Creatures	965
15.2.2 Tuning Steering for Interactivity	965
15.2.3 Steering Behavior Stability	967
15.2.4 Ecosystem Design	967

REFERENCES	971
-------------------	------------

Books, Periodicals, Papers and Websites	971
---	-----

Games	976
-------	-----

INDEX	983
--------------	------------

PART I

AI and Games



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>



INTRODUCTION

GAME DEVELOPMENT lives in its own technical world. It has its own idioms, skills, and challenges. That's one of the reasons games are so much fun to work on. Each game has its own rules, its own aesthetic, its own trade-offs, and the hardware it will run on keeps changing. There's a reasonably good chance you will be the first person to meet and beat a new programming challenge.

Despite numerous efforts to standardize game development, in line with the rest of the software industry (efforts that go back at least 25 years), the style of programming in a game is still rather unique. There is a focus on speed, but it differs from real-time programming for embedded or control applications. There is a focus on clever algorithms, but it doesn't share the same rigor as database server engineering. It draws techniques from a huge range of different sources, but almost without exception modifies them beyond resemblance. And, to add an extra layer of intrigue, developers make their modifications in different ways, often under extreme time pressure, and tailored entirely to the game at hand, leaving algorithms unrecognizable from studio to studio or project to project.

As exciting and challenging as this may be, it makes it difficult for new developers to get the information they need. Twenty years ago, it was almost impossible to get hold of information about techniques and algorithms that real developers used in their games. There was an atmosphere of secrecy, even alchemy, about the coding techniques in top studios. Then came the Internet and an ever-growing range of websites, along with books, conferences, and periodicals. It is now easier than ever to teach yourself new techniques in game development.

This book is designed to help you master one element of game development: artificial intelligence (AI). There have been many articles published about different aspects of game AI: websites on particular techniques, compilations in book form, some introductory texts, and

plenty of lectures at development conferences. But this book was designed to cover it all, as a coherent whole.

In my career I have developed many AI modules for lots of different genres of games. I developed AI middleware tools that have a lot of new research and clever content. I have worked on research and development for next-generation AI, and I get to do a lot with some very clever technologies. However, throughout this book I tried to resist the temptation to pass off how I think it should be done as how it is done. My aim has been to tell it like it is (or for those technologies I mention that are still emerging, to tell you how most people agree it will be).

The meat of this book covers a wide range of techniques for game AI. Some of them are barely techniques, more like a general approach or development style. Some are full-blown algorithms, ready to implement in a reusable way. And others are shallow introductions to huge fields that would fill multiple books on their own. In these cases, I've tried to give enough of an overview for you to understand how and why an approach may be useful (or not).

This book is intended for a wide range of readers: from hobbyists or students looking to get a solid understanding of game AI through to professionals who need a comprehensive reference to techniques they may not have used before. I will assume you can program (though I will not assume any particular language—see [Chapter 13](#) for a discussion of implementation languages), and I will assume you have high-school level mathematical knowledge.

Before we get into the techniques themselves, this chapter introduces AI, its history, and the way it is used. We'll look at a model of AI to help fit the techniques together, and I'll give some background on how the rest of the book is structured.

1.1 WHAT IS AI?

Artificial intelligence is about making computers able to perform the thinking tasks that humans and animals are capable of.

We can program computers to have superhuman abilities in solving many problems: arithmetic, sorting, searching, and so on. Some of these problems were originally considered AI problems, but as they have been solved in more and more comprehensive ways, they have slipped out of the domain of AI developers.

But there are many things that computers aren't good at which we find trivial: recognizing familiar faces, speaking our own language, deciding what to do next, and being creative. These are the domain of AI: trying to work out what kinds of algorithms are needed to display these properties.

Often the dividing line between AI and not-AI is merely difficulty: things we can't do require AI, things we can are tricks and math. It is tempting to get into a discussion of what is 'real' AI, to try defining 'intelligence', 'consciousness' or 'thought'. In my experience, it is an impossible task, largely irrelevant to the business of making games.

In academia, some AI researchers are motivated by those philosophical questions: understanding the nature of thought and the nature of intelligence and building software to model

how thinking might work. Others are motivated by psychology: understanding the mechanics of the human brain and mental processes. And yet others are motivated by engineering: building algorithms to perform human-like tasks. This threefold distinction is at the heart of academic AI, and the different concerns are responsible for different subfields of the subject.

As games developers, we are practical folks; interested in only the engineering side. We build algorithms that make game characters appear human or animal-like. Developers have always drawn from academic research, where that research helps them get the job done, and ignored the rest.

It is worth taking a quick overview of the AI work done in academia to get a sense of what exists in the subject and what might be worth plagiarizing.

1.1.1 ACADEMIC AI

To tell the story, I will divide academic AI into three periods: the early days, the symbolic era, and the natural computing and statistical era. This is a gross oversimplification, of course, and they all overlap to some extent, but I find it a useful distinction. For a collection of papers which give a much more nuanced and fleshed out account, see [38].

The Early Days

The early days include the time before computers, where philosophy of mind occasionally made forays into AI with such questions as: “What produces thought?” “Could you give life to an inanimate object?” “What is the difference between a cadaver and the human it previously was?” Tangential to this was the popular taste in automata, mechanical robots, from the 18th century onward. Intricate clockwork models were created that displayed the kind of animated, animal- or human-like behaviors that we now employ game artists to create in a modeling package (for an academic history and discussion, see [23], and for an in-depth look at two piano playing examples, see [74]).

In the war effort of the 1940s, the need to break enemy codes [8] and to perform the calculations required for atomic warfare motivated the development of the first programmable computers [46]. Given that these machines were being used to perform calculations that would otherwise be done by a person, it was natural for programmers to be interested in AI. Several computing pioneers (such as Turing, von Neumann, and Shannon) were also pioneers in early AI. Turing, in particular, has become an adopted father to the field, as a result of a philosophical paper he published in 1950 [70].

The Symbolic Era

From the late 1950s through to the early 1980s the main thrust of AI research was “symbolic” systems. A symbolic system is one in which the algorithm is divided into two components:

a set of knowledge (represented as symbols such as words, numbers, sentences, or pictures) and a reasoning algorithm that manipulates those symbols to create new combinations that hopefully represent problem solutions or new knowledge.

An expert system, one of the purest expressions of this approach, is among the most famous AI techniques. If today all the AI headlines talk about “deep learning,” in the 1980s, they name dropped “expert systems.” An expert system has a large database of knowledge and it applies a collection of rules to draw conclusions or to discover new things (see [Section 5.8](#)). Other symbolic approaches applicable to games include blackboard architectures (5.9), pathfinding ([Chapter 4](#)), decision trees ([Section 5.2](#)) and state machines ([Section 5.3](#)). These and many more are described in this book.

A common feature of symbolic systems is a trade-off: when solving a problem the more knowledge you have, the less work you need to do in reasoning. Often, reasoning algorithms consist of searching: trying different possibilities to get the best result. This leads us to the golden rule of AI, that we will see in various forms throughout this book:

Search and knowledge are intrinsically linked. The more knowledge you have, the less searching for an answer you need; the more search you can do (i.e., the faster you can search), the less knowledge you need.

It was suggested by researchers Newell and Simon in 1976 that knowledge infused search (known as ‘heuristic search’) is the way all intelligent behavior arises [45]. Unfortunately, despite its having several solid and important features, this theory has been largely discredited as an account of all intelligence. Nevertheless, many people with a recent education in AI are not aware that, as an engineering trade-off, knowledge versus search is unavoidable. Recent work on the mathematics of problem solving has proved this theoretically [76]. At a practical level, AI engineers have always known it.

The Natural Computing / Statistical Era

Through the 1980s and into the early 1990s, there was an increasing frustration with symbolic approaches. The frustration came from various directions.

From an engineering point of view, the early successes on simple problems didn’t seem to scale to more difficult problems. For example: it seemed easy to develop AI that understood (or appeared to understand) simple sentences, but developing an understanding of a full human language seemed no nearer. This was compounded by hype: when AI touted as ‘the next big thing’ failed to live up to its billing, confidence in the whole sector crashed.

There was also an influential philosophical argument that symbolic approaches weren’t biologically plausible. You can’t understand how a human being plans a route by using a symbolic route-planning algorithm any more than you can understand how human muscles work by studying a forklift truck.

The effect was a move toward natural computing: techniques inspired by biology or other natural systems. These techniques include neural networks, genetic algorithms, and simulated

annealing.¹ Many natural computing techniques have been around for a long time. Neural networks, for example, predate the symbolic era; they were first suggested in 1943 [39].

But in the 1980s through to the early 2000s they received the bulk of the research effort. When I began my PhD in artificial intelligence in the 90s, it was difficult to find research places in Expert Systems, for example. I studied genetic algorithms; most of my peers were working on neural networks.

Despite its origin as a correlate to biology, AI research heavily applied mathematics, particularly probability and statistics, to understanding and optimizing natural computing techniques. The ability to handle all the uncertainty and messiness of real-world data, in contrast to the clean and rigid boundaries of the symbolic approaches, led to the development of a wide range of other probabilistic techniques, such as Bayes nets, support-vector machines (SVMs), and Gaussian processes.

The biggest change in AI in the last decade has not come from a breakthrough in academia. We are living in a time when AI is again back in the newspapers: self driving cars, deep fakes, world champion Go programs, and home virtual assistants. This is the era of deep learning. Though many academic innovations are used, these systems are still fundamentally powered by neural networks, now made practical by the increase in computing power.

Engineering

Though newspaper headlines and high-profile applications have flourished in the last five years, AI has been a key technology relevant to solving real-world problems for decades. Navigation systems in cars, job scheduling in factories, voice recognition and dictation, and large-scale search are all more than 20 years old. Google's search technology, for example, has long been underpinned by AI. It is no coincidence that Peter Norvig is both a Director of Research at Google and the co-author (along with his former graduate adviser, professor Stuart Russell) of the canonical undergraduate textbook for modern academic AI [54].

When something is hot, it is tempting to assume it is the only thing that matters. When natural computing techniques took center stage, there was a tendency to assume that symbolic approaches were dead. Similarly, with talk of deep learning everywhere, you might be forgiven for thinking that is what should be used.

But we always come back to the same trade-off: search vs knowledge. Deep learning is the ultimate in the compute intensive search, AlphaGo Zero (the third iteration of the AlphaGo software, [60]) was given very minimal knowledge of the rules of the game, but extraordinary amounts of processing time to try different strategies and learn the best. On the other hand, a character that needs to use a health pack when injured can be told that explicitly:

IF injured THEN use health pack

1. Though old, a good introduction to Genetic Algorithms can be found in [41], and Simulated Annealing in [34]. In their current vogue as Deep Learning, there are many more contemporary resources on neural networks. For a gentle and shallow introduction, see [21], for a more comprehensive guide, [18].

No search required.

The only way any algorithm can outperform another is either to consume more processing power (more search), or to be optimized toward a specific set of problems (more knowledge of the problem).

In practice, engineers work from both sides. A voice recognition program, for example, converts the input signals using known formulae into a format where the neural network can decode it. The results are then fed through a series of symbolic algorithms that look at words from a dictionary and the way words are combined in the language. A statistical algorithm optimizing the order of a production line will have the rules about production encoded into its structure, so it can't possibly suggest an illegal timetable: the knowledge is used to reduce the amount of search required.

Unfortunately, games are usually designed to run on consumer hardware. And while AI is important, graphics have always taken the majority of the processing power. This seems in no danger of changing. For AI designed to run on the device during the game, low computation / high knowledge approaches are often the clear winners. And these are very often symbolic: approaches pioneered in academia in the 70s and 80s.

We'll look at several statistical computing techniques in this book, useful for specific problems. But I have enough experience to know that for games they are often unnecessary: the same effect can often be achieved better, faster, and with more control using a simpler approach. This hasn't changed significantly since the first edition of this book in 2004. Overwhelmingly the AI used in games is still symbolic technology.

1.1.2 GAME AI

Pac-Man [140] was the first game many people remember playing with fledgling AI. Up to that point there had been *Pong* clones with opponent-controlled bats (following the ball up and down) and countless shooters in the *Space Invaders* mold. But *Pac-Man* had definite enemy characters that seemed to conspire against you, moved around the level just as you did, and made life tough.

Pac-Man relied on a very simple AI technique: a state machine (which I'll cover later in [Chapter 5](#)). Each of the four monsters (later called *ghosts* after a disastrously flickering port to the Atari 2600) occupied one of three states: chasing, scattering (heading for the corners at specific time intervals), and frightened (when *Pac-Man* eats a power up). For each state they choose a tile as their target, and turn toward it at each junction. In chase mode, each ghost chooses the target according to a slightly different hard-coded rule, giving them their personalities. A complete breakdown of the *Pac-Man* game mechanics, including the AI algorithms, can be found at [49].

Game AI didn't change much until the mid-1990s. Most computer-controlled characters prior to then were about as sophisticated as a *Pac-Man* ghost.

Take a classic like *Golden Axe* [176] eight years later. Enemy characters stood still (or walked back and forward a short distance) until the player got close to them, whereupon they

homed in on the player. Golden Axe had a neat innovation with enemies that would enter a running state to rush past the player and then switch back to homing mode, attacking from behind. Surrounding the player looks impressive, but the underlying AI is no more complex than Pac-Man.

In the mid-1990s AI began to be a selling point for games. Games like *Beneath a Steel Sky* [174] even mentioned AI on the back of the box. Unfortunately, its much-hyped “Virtual Theater” AI system simply allowed characters to walk backward and forward through the game—hardly a real advancement.

Goldeneye 007 [165] probably did the most to show gamers what AI could do to improve gameplay. Still relying on characters with a small number of well-defined states, Goldeneye added a sense simulation system: characters could see their colleagues and would notice if they were killed (see [Section 11.4](#) for details). Sense simulation was the topic of the moment, with *Thief: The Dark Project* [132] and *Metal Gear Solid* [129] basing their whole game design on the technique.

In the mid-1990s real-time strategy (RTS) games also were beginning to take off. *Warcraft* [85] was one of the first times pathfinding ([Chapter 4](#)) was widely noticed in action (though it had been used several times before). AI researchers were working with emotional models of soldiers in a military battlefield simulation in 1998 when they saw *Warhammer: Dark Omen* [141] doing the same thing. It was also one of the first times people saw robust formation motion ([Section 3.7](#)) in action.

Halo [91] introduced decision trees ([Section 5.2](#)), now a standard method for characters to decide what to do. *F.E.A.R.* [144] used goal oriented action planning (GOAP, see [Section 5.7.6](#)) for the same purpose. With the success of *AlphaGo*, deep learning ([Section 7.9](#)) has become a hot topic, though it is still only practicable off-line.

Some games are designed around the AI. *Creatures* [101] did this in 1997, but games like *The Sims* [136] and its sequels, or *Black & White* [131] have carried on the torch. *Creatures* still has one of the most complex AI systems seen in a game, with a simulated hormone system and a neural network-based brain for each creature (Steve Grand, the game’s designer, describes the technology he created in [19]).

Games like *Half Life* [193] and *The Last of Us* [149] use AI controlled characters to collaborate with the player, meaning they are on screen for much longer, and any faults are much more noticeable.

First person shooters and RTS games have been subjected to significant academic research (there is an annual competition for Starcraft AI, for example). RTS games incorporate AI techniques used in military simulation (to the extent that *Full Spectrum Warrior* [157] started life as a military training simulator).

Sports games and driving games in particular have their own AI challenges, some of which remain largely unsolved (dynamically calculating the fastest way around a race track, for example, would also be helpful to motorsport teams), while role-playing games (RPGs) with complex character interactions still implemented as conversation trees feel overdue for something better (interesting and sophisticated conversation AI has been implemented in games

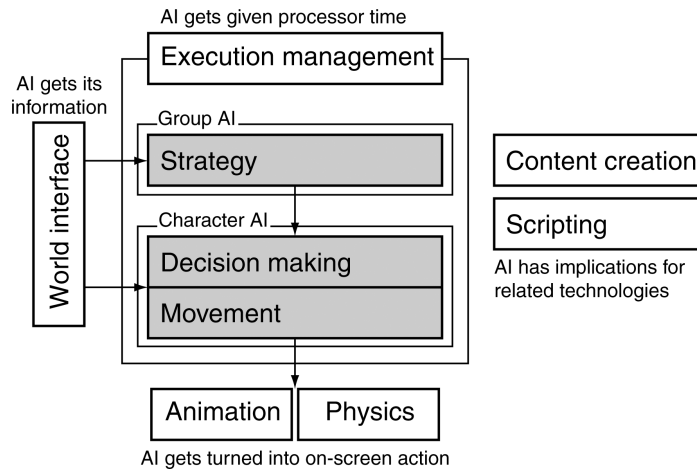


Figure 1.1: The AI model

such as *Façade* [161] and *Blood and Laurels* [130], the one released game using the short lived Versu game engine).

We have come a long way, certainly. But, though we have a massive diversity of AI in games, many genres are still using the simple AI of 1979 because that's all they need.

The AI in most games addresses three basic needs: the ability to move characters, the ability to make decisions about where to move, and the ability to think tactically or strategically. Even though we have a broad range of approaches, they all fulfill the same three basic requirements.

1.2 MODEL OF GAME AI

In this book there is a vast zoo of algorithms and techniques. It would be easy to get lost, so it's important to understand how the bits fit together.

To help, I've used a consistent structure to contextualize the AI used in a game. This isn't the only possible model, and it isn't the only model that would benefit from the techniques in this book. But to make discussions clearer, I will show how each technique fits into a general structure for making intelligent game characters.

Figure 1.1 illustrates this model. It splits the AI task into three sections: movement, decision making, and strategy. The first two sections contain algorithms that work on a character-by-character basis, and the last section operates on a team or side. Around these three AI elements is a whole set of additional infrastructure.

Not all game applications require all levels of AI. Board games like Chess or Risk require

only the strategy level; the characters in the game (if they can even be called that) don't make their own decisions and don't need to worry about how to move.

On the other hand, there is no strategy at all in many games. Non-player characters in a platform game, such as *Hollow Knight* [183], or *Super Mario Bros.* [152] are purely reactive, making their own simple decisions and acting on them. There is no coordination that makes sure the enemy characters do the best job of thwarting the player.

1.2.1 MOVEMENT

Movement refers to algorithms that turn decisions into some kind of motion. When an enemy character without a projectile attack needs to attack the player in *Super Mario Sunshine* [154], it first heads directly for the player. When it is close enough, it can actually do the attacking. The decision to attack is carried out by a set of movement algorithms that home in on the player's location. Only then can the attack animation be played and the player's health be depleted.

Movement algorithms can be more complex than simply homing in. A character may need to avoid obstacles on the way or even work their way through a series of rooms. A guard in some levels of *Splinter Cell* [189] will respond to the appearance of the player by raising an alarm. This may require navigating to the nearest wall-mounted alarm point, which can be a long distance away and may involve complex navigation around obstacles or through corridors.

Lots of actions are carried out using animation directly. If a Sim, in *The Sims*, is sitting by the table with food in front of her and wants to carry out an eating action, then the eating animation is simply played. Once the AI has decided that the character should eat, no more AI is needed (the animation technology used is not covered in this book). If the same character is by the back door when she wants to eat, however, movement AI needs to guide her to the chair (or to some other nearby source of food).

1.2.2 DECISION MAKING

Decision making involves a character working out what to do next. Typically, each character has a range of different behaviors that they could choose to perform: attacking, standing still, hiding, exploring, patrolling, and so on. The decision making system needs to work out which of these behaviors is the most appropriate at each moment of the game. The chosen behavior can then be executed using movement AI and animation technology.

At its simplest, a character may have very simple rules for selecting a behavior. The farm animals in various levels of the *Zelda* games will stand still unless the player gets too close, whereupon they will move away a small distance.

At the other extreme, enemies in *Half-Life 2* [194] display complex decision making, where they will try a number of different strategies to reach the player: chaining together

intermediate actions such as throwing grenades and laying down suppression fire in order to achieve their goals.

Some decisions may require movement AI to carry them out. A melee (hand-to-hand) attack will require the character to get close to its victim. In combat heavy games such as *Dark Souls* [115] decision-making moves the character toward their target, and also determines which attack, and thus which animation, is performed. In other games, once the decision is made, a predetermined animation is played without any additional movement (a Sim eating, for example) or the state of the game is directly modified without any kind of visual feedback (when a country AI in *Sid Meier's Civilization VI* [113] elects to research a new technology, for example, it simply happens with no visual feedback to the player).

1.2.3 STRATEGY

You can go a long way with movement AI and decision making AI, and most action-based three-dimensional (3D) games use only these two elements. But to coordinate a whole team, some strategic AI is required.

In the context of this book, strategy refers to an overall approach used by a group of characters. In this category are AI algorithms that don't control just one character, but influence the behavior of a whole set of characters. Each character in the group may (and usually will) have their own decision making and movement algorithms, but overall their decision making will be influenced by a group strategy.

In the original *Half-Life* [193], enemies worked as a team to surround and eliminate the player. One would often rush past the player to take up a flanking position. This has been followed in more recent games, such as the evolving AI engine in the *Medal of Honor* [78] franchise. Over time we have seen increasing sophistication in the kinds of strategic actions that a team of enemies can carry out.

1.2.4 INFRASTRUCTURE

AI algorithms on their own are only half of the story, however. In order to actually build AI for a game, we'll need a whole set of additional infrastructure. The movement requests need to be turned into action in the game by using either animation or, increasingly, physics simulation.

Similarly, the AI needs information from the game to make sensible decisions. This is sometimes called “perception” (especially in academic AI): working out what information the character knows. In practice, it is much broader than just simulating what each character can see or hear, but includes all interfaces between the game world and the AI. This world interfacing is often a significant proportion of the work done by an AI programmer, and in my experience it is a large proportion of the AI debugging effort.

Finally, the whole AI system needs to be managed so it uses the right amount of processor time and memory. While some kind of execution management typically exists for each area

of the game (level of detail algorithms for rendering, for example), managing the AI raises a whole set of techniques and algorithms of its own.

Each of these components may be thought of as being out of the remit of the AI developer. Sometimes they are (in particular, the animation system is often part of the graphics engine, or increasingly has its own dedicated programmers), but they are so crucial to getting the AI working that they can't be avoided altogether. In this book I have covered each infrastructure component except animation in some depth.

1.2.5 AGENT-BASED AI

I don't use the term "agents" very much in this book, even though the model I've described is an agent-based model.

In this context, agent-based AI is about producing autonomous characters that take in information from the game data, determine what actions to take based on the information, and carry out those actions.

It can be seen as bottom-up design: you start by working out how each character will behave and by implementing the AI needed to support that. The overall behavior of the game is then a function of how the individual character behaviors work together. The first two elements of the AI model we'll use, movement and decision making, make up the AI for an agent in the game.

In contrast, a non-agent-based AI seeks to work out how everything ought to act from the top down and builds a single system to simulate everything. An example is the traffic and pedestrian simulation in the cities of *Grand Theft Auto 3* [104]. The overall traffic and pedestrian flows are calculated based on the time of day and city region and are only turned into individual cars and people when the player can see them.

The distinction is hazy, however. We'll look at level of detail techniques that are very much top down, while most of the character AI is bottom up. A good AI developer will mix and match any reliable techniques that get the job done, regardless of the approach. That pragmatic approach is the one I always follow. So in this book, I avoid using agent-based terminology, and prefer to talk about game characters in general, however they are structured.

1.2.6 IN THE BOOK

In the text of the book each chapter will refer back to this model of AI, pointing out where that topic fits. The model is useful for understanding how things connect and which techniques are alternatives for others.

But the dividing lines aren't always sharp; this is intended to be a general model, not a straitjacket. In the final game code there are no joins. The whole set of AI techniques from each category, as well as a lot of the infrastructure, will all operate seamlessly together.

Many techniques fulfill roles in more than one category. Pathfinding, for example, can be both a movement and a decision making technique. Similarly, some tactical algorithms that

analyze the threats and opportunities in a game environment can be used as decision makers for a single character or to determine the strategy of a whole team.

1.3 ALGORITHMS AND DATA STRUCTURES

There are three key elements to implementing the techniques described in this book: the algorithm itself, the data structures that the algorithm depends on, and the way the game world is represented to the algorithm (often encoded as an appropriate data structure). Each element is dealt with separately in the text.

1.3.1 ALGORITHMS

Algorithms are step-by-step processes that generate a solution to an AI problem. We will look at algorithms that generate routes through a game level to reach a goal, algorithms that work out which direction to move to intercept a fleeing enemy, algorithms that learn what the player should do next, and many others.

Data structures are the other side of the coin to algorithms. They hold data in such a way that an algorithm can rapidly manipulate it to reach a solution. Often, data structures need to be tuned for one particular algorithm, and their execution speeds are intrinsically linked.

You will need to know a set of elements to implement and tune an algorithm, and these are treated step by step in the text:

- The problem that the algorithm tries to solve
- A general description of how the solution works, including diagrams where they are needed
- A pseudo-code presentation of the algorithm
- An indication of the data structures required to support the algorithm, including pseudo-code, where required
- Implementation advice, where needed
- Analysis of the algorithm performance: its execution speed, memory footprint, and scalability
- Weaknesses in the approach

Often, I will present a set of algorithms that gets increasingly more complex and powerful. The simpler algorithms are presented to help you get a feeling for why the complex algorithms have their structure. The stepping-stones are described a little more sketchily than the full system.

Some of the key algorithms in game AI have literally hundreds of variations. This book can't hope to catalog and describe them all. When a key algorithm is described, I will often give a quick survey of the major variations in briefer terms.

Performance Characteristics

To the greatest extent possible, I have tried to include execution properties of the algorithm in each case. Execution speed and memory consumption often depend on the size of the problem being considered. I have used the standard $O()$ notation to indicate the order of the most significant element in this scaling.

An algorithm might be described as being $O(n \log n)$ in execution and $O(n)$ in memory, where n is usually some kind of component of the problem, such as the number of other characters in the area, or the number of power-ups in the level.

A good text on general algorithms for computer science (e.g. [9] or [57]) will give a full mathematical treatment of how $O()$ values are arrived at and the implications they have for the real-world performance of an algorithm. In this book I omit derivations or proofs. Where a complete indication of the complexity is too involved, I indicate the approximate running time or memory in the text, rather than attempt to derive an accurate $O()$ value.

Some algorithms have misleading performance characteristics. It is possible to set up highly improbable situations to deliberately make them perform poorly. In regular use (and certainly in any use you're likely to have in a game), they will have much better performance. When this is the case, I have tried to indicate both the expected and the worst case results. Unless otherwise noted, you can probably ignore the worst case value safely.

Pseudo-Code

Algorithms in this book are presented in pseudo-code for brevity and simplicity. Pseudo-code is an imaginary programming language that cuts out any implementation details particular to any real programming language. It should describe the algorithm in sufficient detail so you can implement it in the language of your choice. The pseudo-code in this book has more of a procedural programming language feel compared to the pseudo-code you would find in more theoretical books on algorithms. In addition, I use pseudo-code to describe data structures and interfaces, as well as algorithms, because the algorithms in this book are often intimately tied to surrounding bits of software in a way that is more naturally captured with code.

Many AI algorithms need to work with relatively sophisticated data structures: lists, tables, priority queues, associative arrays, and so on. Some languages provide these built-in, others make them available as libraries, or accessed through functions. To make what is going on clearer, the pseudo-code treats these data structures as if they were part of the language, simplifying the code significantly.

When creating the pseudo-code in this book, I've stuck to these conventions, where possible:

- Indentation indicates block structure and is normally preceded by a colon. There are no including braces or "end" statements. This makes for much simpler code, with less redundant lines to bloat the listings. Good programming style always uses indentation as well as other block markers, so we may as well just use indentation.

- Functions are introduced by the keyword `function`, and classes are introduced by the keyword `class`. An inherited class specifies its parent class after the keyword `extends`. The pseudo-code will not include setters and getters, unless they are significant, all member variables will be accessible.
- Function parameters are included in parentheses, both when the function is defined and when it is called. Class methods are accessed by name using a period between the instance variable and the method—for example, `instance.variable()`.
- Types may be given after a variable name or parameter name, separated by a colon. The return type from a function follows `->`.
- All variables are local to the scope where they are declared, usually the function or method. Variables declared within a class definition, but not in a method, are class instance variables.
- The single equal sign `=` is an assignment operator, whereas the double equal sign `==` is an equality test. Assignment modifiers such as `+=` exist for all mathematical operations.
- Looping constructs are `while` and `for`. The `for` loop can iterate over any array. It can also iterate over a series of numbers, using the syntax `for a in 0..5`.
- A range is indicated by `0..5`. Ranges always include their lowest value, but not their highest, so `1..4` includes the numbers (1, 2, 3) only. Ranges can be open, such as `1..`, which is all numbers greater than or equal to 1; or `..4`, which is identical to `0..4`. Ranges can be decreasing, but notice that the highest value is still not in the range: so `4..0` is the set (3, 2, 1, 0).²
- Boolean operators are `and`, `or`, and `not`. Boolean values can be either `true` or `false`.
- The symbol `#` introduces a comment for the remainder of the line.
- Array elements are given in square brackets and are zero indexed (i.e., the first element of array `a` is `a[0]`). A sub-array is signified with a range in brackets, so `a[2..5]` is the sub-array consisting of the 3rd to 5th elements of the array `a`. Open range forms are valid: `a[1..]` is a sub-array containing all but the first element of `a`.
- In general, I assume that arrays are equivalent to lists. We can write them as lists and freely add and remove elements.

As an example, the following sample is pseudo-code for a simple algorithm to select the highest value from an unsorted array:

2. The justification for this interpretation is connected with the way that loops are normally used to iterate over an array. Indices for an array are commonly expressed as the range `0..length(array)`, in which case we don't want the last item in the range. If we are iterating backward, then the range `length(array)..0` is similarly the one we need. I was undecided about this interpretation for a long time, but felt that the pseudo-code was more readable if it didn't contain lots of “-1” values. Usually this only happens when iterating over all the indices of an array, in which case the code looks correct. When I feel it may be ambiguous, I will clarify my intention with a note.

```

1 function maximum(array:float[]) -> float:
2     max: float = array[0]
3     for element in array[1..]:
4         if element > max:
5             max = element
6     return max

```

Occasionally, an algorithm-specific bit of syntax will be explained as it arises in the text.

Programming polymaths will probably notice that the pseudo-code has more than a passing resemblance to the Python programming language, with Ruby-like structures popping up occasionally and a seasoning of Lua. This is deliberate, insofar as Python is an easy-to-read language. Nonetheless, the similarity is only skin deep, listings are still pseudo-code and not Python implementations. The similarity is not supposed to suggest a language or an implementation bias.³

1.3.2 REPRESENTATIONS

Information in the game often needs to be turned into a suitable format for use by the AI. Often, this means converting it to a different representation or data structure. The game might store the level as meshes of 3D geometry and the character positions as (x, y, z) locations in the world.

Converting between these representations is a critical process because it often loses information (that's the point: to simplify out the irrelevant details), and you always run the risk of losing the wrong bits of data.

Choosing the correct representation is a key element of implementing AI, and certain representations are particularly important in game AI. Several of the algorithms in the book require the game to be presented to them in a particular format.

Although very similar to a data structure, we will often not worry directly about how the representation is implemented, but instead will focus on the interface it presents to the AI code. This makes it easier for you to integrate the AI techniques into your game, simply by creating the right glue code to turn your game data into the representation needed by the algorithms.

For example, imagine we want to work out if a character feels healthy or not as part of some algorithm for determining its actions. We might simply require a representation of the character with a method we can call:

3. In fact, while Python is a good language for rapid prototyping, and is widely used in the toolchains of game development, it is usually too slow for building the core AI engine in a production game. Languages such as Python, JavaScript, and particularly Lua are sometimes used as scripting languages in a game, and I'll cover their use in that context in [Chapter 13](#).

```

1 class Character:
2     # Return true if the character feels healthy, and false otherwise.
3     function feelsHealthy() -> bool

```

You may then implement this by checking against the character's health score, by keeping a Boolean “healthy” value for each character, or even by running a whole algorithm to determine the character's psychological state and its perception of its own health. As far as the decision making routine is concerned, it doesn't matter how the value is being generated.

The pseudo-code defines an interface (in the object-oriented sense) that can be implemented in any way you choose.

When a representation is particularly important or tricky (and there are several that are), I will describe possible implementations in some depth.

1.3.3 IMPLEMENTATION

Even a decade ago, most developers used C++ for their AI code. A decade before that a significant number relied on C. Games development is much more varied now. Swift and Java (for mobile platforms), C# for the unity game engine, JavaScript on the web. There are many other languages used here and there in game development: Lisp, Lua, or Python, particularly as scripting languages; ActionScript for the few remaining Flash developers. I've personally worked with all these languages at one point or another, so I've tried to be as language independent as possible, while still giving some advice on implementation.

Of these, C and C++ are still used for code that absolutely, positively has to run as fast as possible. In places some of the discussion of data structures and optimizations will focus on C++, because the optimizations are C++ specific.

1.4 LAYOUT OF THE BOOK

This book is split into five sections.

[Part I](#) introduces AI and games in this chapter and [Chapter 2](#), giving an overview of the book and the challenges that face the AI developer in producing interesting game characters.

[Part II](#) is the meat of the technology in the book, presenting a range of different algorithms and representations for each area of our AI model. It contains chapters on decision making ([Chapter 5](#)) and movement ([Chapter 3](#)) and a specific chapter on pathfinding ([Chapter 4](#), a key element of game AI that has elements of both decision making and movement). It also contains information on tactical and strategic AI ([Chapter 6](#)), including AI for groups of characters. There is a chapter on learning ([Chapter 7](#)), a key frontier in game AI, and finally chapters on procedural content generation ([Chapter 8](#)) and board game AI ([Chapter 9](#)). None of these chapters attempts to connect the pieces into a complete game AI. It is a pick and mix array of techniques that can be used to get the job done.

[Part III](#) looks at the technologies that enable the AI to do its job. It covers everything

from execution management ([Chapter 10](#)) to world interfacing ([Chapter 11](#)), getting the game content into an AI-friendly format ([Chapter 12](#)), and the programming languages used to code your AI ([Chapter 13](#)).

[Part IV](#) looks at designing AI for games. It contains a genre-by-genre breakdown of the way techniques are often combined to make a full game ([Chapter 14](#)). If you are stuck trying to choose among the range of different technique options, you can look up your game style here and see what is normally done (then do it differently, perhaps). It also looks at a handful of AI-specific game genres that seek to use the AI in the book as the central gameplay mechanic ([Chapter 15](#)).

Finally, appendices provide references to other sources of information.



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>



2

GAME AI

BEFORE going into detail with particular techniques and algorithms, it is worth spending a little time thinking about what we need from our game's AI. This chapter looks at the high-level issues around game AI: what kinds of approaches work, what they need to take account of, and how they can all be put together.

2.1 THE COMPLEXITY FALLACY

It is a common mistake to think that the more complex the AI in a game, the better the characters will look to the player. Creating good AI is all about matching the requirements of the game to the right behaviors and the right algorithms to produce them. There is a bewildering array of techniques in this book, and the right one isn't always the most obvious choice.

Countless examples of difficult to implement, complex AI have resulted in poor, or even stupid looking behavior. Equally, a very simple technique can be perfect, when used well.

2.1.1 WHEN SIMPLE THINGS LOOK GOOD

In the last chapter I mentioned *Pac-Man* [140], one of the first games with any form of character AI. The AI has three states: one normal state when the player is collecting pips; a second state when the player has eaten the power-up and is out for revenge; and a final state that triggers at timed intervals, to have the ghosts back off a little.

In all three states, each of the four ghosts has a target. It moves in a straight line until it reaches a junction, then chooses whichever route is closest to the direction of its target. It

doesn't attempt to plan the entire route, or even check its target can be reached, it just moves toward it. In their chasing state, when hunting the player, each ghost has its own simple snippet of code to choose a target. Blinky (the red ghost) always targets the player position. Pinky (the pink one, obviously) targets a square four spaces in front of the player, even if that is inside or on the other side of a wall. Inky (light blue) uses a modified offset of its own and the player's position. Clyde (orange) targets the player if they are far away, or the corner of the board if close. All these targeting routines can be implemented in a line or two of code.

This is about as simple as you can imagine an AI for a moving character. Any simpler and the ghosts would be either very predictable (if they always homed in) or purely random. On their own, the ghosts strategy can be easily predicted; their AI does not pose a challenge. But together, the different behaviors of each ghost are enough to make a significant opposing force—so much so that the AI to this day gets flattering comments. For example, this comment recently appeared on a website: “To give the game some tension, some clever AI was programmed into the game. The ghosts would group up, attack the player, then disperse. Each ghost had its own AI.”

Other players have reported strategies among the ghosts: “The four of them are programmed to set a trap, with Blinky leading the player into an ambush where the other three lie in wait.”

A simple AI, done well, can appear to a player to be much more intelligent than it is.

The same thing has been reported by many other developers on own their games. As an extreme example: a few years ago Chris Kingsley of Rebellion mentioned an unpublished Nintendo Game Boy title in which particular enemy characters simply home in on the player, but for variation, sidestep at random intervals as they move forward. Players reported that these characters ‘anticipated’ their firing patterns and dodged out of the way.

The AI wasn't anticipating anything, and only managed to dodge by coincidence some of the time. But a timely sidestep at a crucial moment stayed in the player's mind and shaped their perception of the AI.

2.1.2 WHEN COMPLEX THINGS LOOK BAD

Of course, the opposite thing can easily happen. A game that I looked forward to immensely was *Herdy Gerdy* [97], one of the launch games Sony used to tout the new gameplay possibilities of the ‘emotion engine’ chip in their PlayStation 2 hardware. The game is a herding game. An ecosystem of characters is present in the game level. The player has to herd individuals of different species into their corresponding pens. Herding had been used before and has since as a component of a bigger game, but in *Herdy Gerdy* it constituted all of the gameplay. There is a section on AI for this kind of game in [Chapter 15](#).

Unfortunately, the characters' movement AI was not quite up to the challenge of its rich level design. It was easy to get them caught on the scenery, and their collision detection could leave them stuck in irretrievable places. This would be frustrating (though not uncommon)

in any game. But it interacted with the herding AI in a way that made some characters appear unexpectedly unintelligent. Reviews were mixed, and sales lackluster.

Unlike Herdy Gerdy, *Black & White* [131] achieved significant sales success. But at places it also suffered from great AI looking bad. The game involves teaching a character what to do by a combination of example and feedback. When people first play through the game, they often end up inadvertently teaching the creature bad habits, and in the worst cases it may become unable to carry out even the most basic actions. By paying more attention to how the creature's AI works, players are able to manipulate it better, but the illusion of teaching a real creature can be gone.

Most of the complex things I've seen that looked bad never made it to the final game. It is a perennial temptation for developers to use the latest techniques and the most hyped algorithms to implement their character AI. Late in development, when a learning AI still can't learn how to steer a car around a track without driving off at every corner, simpler algorithms come to the rescue and make it into the game's release.

Knowing when to be complex and when to stay simple is the most difficult element of the game AI programmer's art. The best AI programmers are those who can use a very simple technique to give the illusion of complexity.

This is easier when there is a tight feedback loop between implementation and game design. A slight modification to the requirements can mean a better AI technique can be used, which leads to a better game. Sometimes this means making the required behavior simpler, to make it much more robust. Unfortunately, with the large team sizes on mass-market PC and console games, it is difficult for a programmer to have much influence. Indie and mobile games, whose teams are much smaller, though not as small as they were a few years ago, still have more opportunity.

2.1.3 THE PERCEPTION WINDOW

Unless your AI is controlling an ever-present sidekick or a one-on-one enemy, chances are your player will only come across a character for a short time.

This can be a significantly short time for disposable guards whose life purpose is to be shot. More difficult enemies can be on-screen for a few minutes as their downfall is plotted and executed.

When we seek to understand someone in real life, we naturally put ourselves into their shoes. We look at their surroundings, the information they are gleaned from their environment, and the actions they are carrying out. The same happens with game characters. A guard standing in a dark room hears a noise: "I'd flick the light switch," we think. If the guard doesn't do that, we might assume they are stupid.

If we only catch a glimpse of someone for a short while, we don't have enough time to understand their situation. If we see a guard who has heard a noise suddenly turn away and move slowly in the opposite direction, we assume the AI is faulty. The guard should have moved across the room toward the noise. If we observed them for longer to see the guard

head over to a light switch by the exit, their action would be understandable. Then again, the guard might not flick on the light switch after all, and we might take that as a sign of poor implementation. But the guard may know that the light is inoperable, or they may have been waiting for a colleague to slip some cigarettes under the door and thought the noise was a predefined signal. If we knew all that, we'd know the action was intelligent after all.

This no-win situation is the perception window. You need to make sure that a character's AI matches its purpose in the game and the attention it will get from the player. Adding more AI to incidental characters might endear you to the rare gamer who spends hours analyzing each level, checking for curious behavior or bugs, but everyone else (including the publisher and the press) may think your programming was sloppy.

2.1.4 CHANGES OF BEHAVIOR

The perception window isn't only about time. Think about the ghosts in Pac-Man again. They might not give the impression of sentience, but they don't do anything out of place. This is because they rarely change behavior (the most noticeable is their transformation when the player eats a power-up).

Whenever a character in a game changes behavior, the change is far more conspicuous than the behavior itself. In the same way, when a character's behavior should obviously change and doesn't, it draws attention. If two guards are standing talking to each other and you shoot one down, the other guard shouldn't carry on the conversation!

A change in behavior almost always occurs when the player is nearby or has been spotted. This is the same in platform games as it is in real-time strategy. A good solution is to keep only two behaviors for incidental characters—a normal action and a player-spotted action.

2.2 THE KIND OF AI IN GAMES

Games have always come under criticism for being poorly programmed, in a software engineering sense: they use tricks, arcane optimizations, and unproven technologies to get extra speed or neat effects. Though game engines may be reused, gameplay code usually isn't (or at least isn't written with that in mind) and strong time pressures mean programmers often do whatever they need to get the game done. Game AI is no different.

There is a big gulf between what qualifies as AI in games (i.e. what is the responsibility of an AI programmer) and what the rest of the programming industry or academia considers to be AI.

In my experience, AI for a game is equal parts hacking (*ad hoc* solutions and neat effects), heuristics (rules of thumb that only work in most, but not all, cases), and algorithms (the "proper" stuff). Most of this book is aimed at the last group, because those are the techniques we can generalize, examine analytically, use in multiple games, and build into an AI engine.

But *ad hoc* solutions and heuristics are just as important and can breathe as much life into characters as the most complicated algorithm.

2.2.1 HACKS

There's a saying that goes "If it looks like a fish and swims like a fish, it's probably a fish." We understand a behavior by replicating it to a sufficient accuracy.

As a psychological approach it has some pedigree (it is related to the behaviorist school of psychology) but has been largely superseded. This fall from fashion has influenced psychological approaches to AI, as well. At one point it was quite acceptable to learn about human intelligence by making a machine to replicate it, but now that is considered poor science. And with good reason; after all, building a machine to play skillful Chess involves algorithms that evaluate millions of board positions. Human beings are simply not capable of this.

On the other hand, as AI engineers, we are not paid to be interested in the nature of reality or mind; we want characters that look right. In most cases, this means starting from human behaviors and trying to work out the easiest way to implement them in software.

Good AI in games usually works in this direction. Developers rarely build a great new algorithm and then ask themselves, "so what can I do with this?" Instead, you start with a design for a character and apply the most relevant tool to get the result.

This means that what qualifies as game AI may be unrecognizable as an AI technique. A simple random number generator applied judiciously can produce a lot of believability. Generating a random number isn't an AI technique as such. In most languages there are built-in functions to get a random number, so there is certainly no point giving an algorithm for it! But it can work in a surprising number of situations.

Another good example of creative AI development is *The Sims* [136]. While there are reasonably complicated things going on under the surface, a lot of the character behavior is communicated with animation. Remove the character animation, and the AI would look far less impressive. In *Star Wars: Episode 1 – Racer* [133], characters who are annoyed gave a little sideswipe to other characters. *Quake II* [123] introduced the "gesture" command, now used in a vast range of first person games, where characters (and players) can flip their enemy off. All these require no significant AI infrastructure. They don't need complicated cognitive models, learning, or neural networks. They just need a few lines of code that schedule an animation at the appropriate time.

Always be on the lookout for simple things that can give the illusion of intelligence. If you want engaging emotional characters, is it possible to add a couple of emotion animations (a frustrated rub of the temple, perhaps, or a stamp of the foot) to your game? Triggering these in the right place is much easier than trying to represent the character's emotional state through their actions. Do you have a portfolio of behaviors that the character will choose from? Will the choice involve complex weighing of many factors? If so, it might be worth trying a version of the AI that picks a behavior purely at random (maybe with different probabilities for each behavior). *You* might be able to tell the difference, but your customers may not; so try it out before you code the complex version.

2.2.2 HEURISTICS

A heuristic is a rule of thumb, an approximate solution that might work in many situations but is unlikely to work in all.

Human beings use heuristics all the time. We don't try to work out all the consequences of our actions. Instead, we rely on general principles that we've found to work in the past (or that we have been taught or even brainwashed with, equally). It might range from something as simple as "if you lose something then retrace your steps to look for it" to heuristics that govern our life choices, such as "never trust a used-car salesman."

Heuristics have been codified and incorporated into some of the algorithms in this book, and saying "heuristic" to an AI programmer often conjures up images of pathfinding or goal-oriented behaviors. Beyond these, many of the techniques in this book rely on heuristics that may not always be explicit. There is a trade-off between speed and accuracy in areas such as decision making, movement, and tactical thinking (including board game AI). When accuracy is sacrificed, it is usually by replacing search for a correct answer with a heuristic.

A wide range of heuristics can be applied to general AI problems that don't require a particular algorithm.

In our perennial Pac-Man example, the ghosts move by taking the route at a junction that leads toward their current target. They don't attempt to calculate the best route: either the shortest or fastest. That might be quite complex and involve turning back on oneself, and it might be ultimately redundant as the position of the player continues to change. But the rule of thumb (move in the current direction of the target) works most of the time and provides sufficient competence for the player to understand that the ghosts aren't purely random in their motion.

In *Warcraft* [85] (and many other RTS games that followed) there is a heuristic that moves a character forward slightly so they can engage an enemy standing a fraction beyond the character's reach. While this worked in most cases, it wasn't always the best option. Many players got frustrated as comprehensive defensive structures went walkabout when enemies came close. Later, RTS games allowed the player to choose whether this behavior was switched on or not.

In many strategic games, including board games, different units or pieces are given a single numeric value to represent how "good" they are. In Chess, pawns are often given one point, bishops and knights three, rooks five, and the queen eight. This is a heuristic; it replaces complex calculations about the capabilities of a unit with a single number. And the number can be defined by the programmer in advance. The AI can work out which side is ahead simply by adding the numbers. In an RTS it can find the best value offensive unit to build by comparing the number with the cost. A lot of useful effects can be achieved just by manipulating the number.

There isn't an algorithm or a technique for this. And you won't find it in published AI research. But it is the bread and butter of an AI programmer's job.

Common Heuristics

A handful of heuristics appear over and over. They are good starting points when initially tackling a problem.

Most Constrained

Given the current state of the world, one item in a set needs to be chosen. The item chosen should be the one that would be an option for the fewest number of states.

For example, a squad of characters engages a group of enemies. One of the enemies is wearing a type of armor that only one rifle can penetrate. One squad member has this rifle. When they select who to attack, the most constrained heuristic comes into play; only one squad member can attack this enemy, so that is the action that they should take. Even if their weapon would be more powerful against different enemy, their squad mates should handle the others.

Do the Most Difficult Thing First

The hardest thing to do often has implications for lots of other actions. It is better to do this first, rather than find that the easy stuff goes well but is ultimately wasted.

For example, an army has two squads with empty slots. The computer schedules the creation of five Orc warriors and a huge Stone Troll. It wants to end up with balanced squads. How should it assign the units to squads? The Stone Troll requires the most slots, so is the hardest to assign. It should be placed first.

If the Orcs were assigned first, they would be balanced between the two squads, leaving room for half a Troll in each squad, but nowhere for the Troll to go.

Try the Most Promising Thing First

If there are a number of options open to the AI, it is often possible to give each one a really rough-and-ready score. Even if this score is dramatically inaccurate, trying the options in decreasing score order will provide better performance than trying things purely at random.

2.2.3 ALGORITHMS

And so we come to the final third of the AI programmer's job: building algorithms to support interesting character behavior. Hacks and heuristics will get you a long way, but relying on them solely means you'll have to constantly reinvent the wheel. General bits of AI, such as movement, decision making, and tactical thinking all benefit from tried and tested methods that can be endlessly reused.

This book is about these kinds of technique, and the next few chapters introduces a large number of them. Just remember that for every situation where a complex algorithm is the best way to go, there are likely to be several more where a simpler hack or heuristic will get the job done.

2.3 SPEED AND MEMORY CONSTRAINTS

The biggest constraint on the AI developer's job is the physical limitations of the machine. Game AI doesn't have the luxury of days of processing time and terabytes of memory. We don't even have the luxury of using all the processor and memory of the computer the game is running on. Other tasks need space and time, such as graphics, sound, networking, and input. In teams where different groups of developers have to work on their specialties in parallel, a speed and memory budget will be set.

One of the reasons AI techniques from academia or commercial research don't achieve widespread use is their processing time or memory requirements. An algorithm that might be compelling in a simple demo can slow a production game to a standstill.

This section looks at low-level hardware issues related to the design and construction of AI code. Most of what is contained here is general advice for all game code. If you're up to date with current game programming issues and just want to get to the AI, you can safely skip this section.

2.3.1 PROCESSOR ISSUES

The most obvious limitation on the efficiency of a game is the speed of the processor on which it is running. Originally all the games machines had a single main processor, which was also responsible for graphics. Most game hardware now has several CPUs (several processing cores on the same piece of silicon, usually), and dedicated GPUs for processing graphics.

As a general rule, CPUs are faster and more flexible, where a GPU is more parallel. When the task can be split up into many simple subtasks, all running at the same time, the tens up to thousands of processing cores on a GPU can be orders of magnitude faster than the same task running sequentially on the CPU.

Graphics card drivers used to have 'fixed function' pipelines, where the graphics code was built into the driver, and could only be tweaked within narrow parameters. It was impossible to do very much other than graphics on the graphics card. Now drivers support technologies such as Vulkan, DirectX 11, CUDA, and OpenCL, which allow general-purpose code to be executed on the GPU. As a result, more functionality has been moved to the GPU, freeing up more processing power on the CPU.

The share of the processing time dedicated to AI has grown in fits and starts over the last two decades. In some cases now making up most of the CPU load, with some AI running on the GPU. Along with the increase in processor speeds, this is obviously good news for AI developers wanting to apply more complicated algorithms, particularly to decision making and strategizing. But, while incremental improvements in processor time help unlock new techniques, they don't solve the underlying problem. Many AI algorithms take a long time to run. A comprehensive pathfinding system can take tens of milliseconds to run per character. Clearly, in an RTS with 1000 characters, there is no chance of running each frame.

Complex AI that does work in games needs to be split into bite-size components that

can be distributed over multiple frames. The chapter on resource management shows how to accomplish this. Applying these techniques to many long running AI algorithms can bring them into the realm of practicality.

Low-Level Concerns

One large change in the industry in the last 10 years has been the move away from C++ having a hegemony over game programming. Now, character behavior, game logic, and AI is often written in higher-level languages such as C#, Swift, Java, or even scripting languages. This is significant because these languages provide the programmer less ability to micromanage the performance characteristics of their code. There are AI programmers still working in C++, who still need a good knowledge of the ‘bare metal’ performance characteristics of the processors, but in my recent experience such programmers tend to be low-level specialists working on AI engines: portfolios of functionality designed to be reused in multiple games.

In the first and second edition of this book, I described three low-level issues in detail: SIMD, superscalar architectures, and virtual functions. In this edition I will describe them only briefly. For reasons I will describe below, in my own professional practice, I haven’t been directly concerned with any of these for several years.

SIMD (single instruction, multiple data) are a set of registers on modern hardware large enough to fit several floating point numbers. Mathematical operators can be applied to these registers, which has the effect of running the same code against multiple pieces of data in parallel. This can dramatically speed up some code, particularly geometric reasoning. Although CPUs have dedicated registers for SIMD, they provide the best speed up on code that tends to suit the GPU. Optimizing for SIMD on the CPU is often redundant, when the code can be moved onto the GPU.

Superscalar CPUs have several execution paths active at the same time. Code is split among the parts to execute in parallel and the results are then recombined into the final result. When the result of one pipeline depends on another, this can involve either waiting, or guessing what the result might be and redoing the work if it proves to be wrong (known as *branch prediction*). In the last decade, multi-core CPUs, where several independent CPUs allow different threads to run in parallel, have become almost ubiquitous. Although each core may still be superscalar, this is now largely treated as a behind-the-scenes detail, irrelevant to AI programmers. Better speed ups can be attained by concentrating on making AI code parallelizable, rather than worrying about the details of branch prediction.

AI code can take advantage of this parallelism by either running AI for different characters in different threads, or by running all AI in a different thread to other game systems. These threads will then be executed in parallel on different cores, when available. Multiple threads doing the same thing (such as one character running its AI in each thread) is often more performant, as it is easier to make sure all processors are being used to the same capacity, and more flexible, as it scales without rebalancing to hardware with a different number of cores.

When AI routinely had to be careful of every single CPU cycle used, it was common to

avoid virtual classes in C++, with their virtual function call overhead. This meant avoiding object-oriented polymorphism, whenever possible. A virtual function call stores the memory location in which the function is implemented in a variable (in a structure called a function table, or vtable). Calling a function therefore involves looking up the variable at runtime, and then looking up the location that variable specifies. Though this extra look-up uses a trivial amount of time, it could interact significantly with the branch predictor and processor cache. For this reason virtual functions, and hence polymorphism, had a bad reputation. A reputation that has largely faded in the past 10 years. The code in this book has always been written in a polymorphic style. Now, game engines such as Unity, Unreal, Lumberyard, and Godot assume that game logic will be polymorphic.

2.3.2 MEMORY CONCERNS

Most AI algorithms do not require a large amount of RAM, often just a few, up to tens of megabytes. This small storage requirement, easily achievable on modest mobile devices, is ample for heavyweight algorithms such as terrain analysis and pathfinding. Massively multiplayer online games (MMOGs) typically require much more storage for their larger worlds, but are run on server farms where sufficient memory can be installed (even then, we are only talking gigabytes of RAM, rarely more). Huge worlds are usually sharded into separate sections, or else characters are limited to certain areas, further reducing the AI memory requirements.

So it is not the amount of memory which is usually the limiting factor, but the way it is used. Allocation, and cache coherence are both memory concerns that affect performance. They can both influence the implementation of an AI algorithm.

Allocation and Garbage Collection

Allocation is the process of asking for memory in which to place data. When that memory is no longer needed it is said to be freed or deallocated. Allocation and deallocation are relatively fast, as long as memory is available.

Low-level languages such as C require the programmer to free memory manually. Languages such as C++ and Swift, when memory is allocated for a particular object, provide ‘reference counting.’ This stores the number of places that know about the existence of the object. When the object is no longer referenced, the counter drops to 0, and the memory is freed. Unfortunately, both these approaches can mean that memory that should be freed is never freed. Either the programmer forgets to free manually, or there is a circular set of references, such that their counters never drop to 0. Many higher-level languages implement sophisticated algorithms to collect this ‘garbage’, i.e., free memory that is no longer useful. Unfortunately garbage collection can be expensive. In languages such as C#, particularly in the mono runtime that runs the Unity game engine, garbage collection can be slow enough to delay a rendering frame, causing a visual stutter. This is unacceptable to most developers.

As a result, implementing AI algorithms for higher-level languages often involves trying not to allocate and deallocate objects while a level is running. The data required for the entire level is reserved when the level begins, and only released when the level ends. Several of the algorithms in this book assume that new objects can be created at any time, and will be left to disappear when no longer needed. On a platform with time-consuming garbage collection, it may be important to modify these implementations. For example: several of the pathfinding algorithms (in [Chapter 4](#)) create and store data for each location in the map, when that location is first considered. When the path is complete, none of the intermediate location data are needed. A garbage-collection-friendly implementation might create a single pathfinding object, containing data for every location in the map. That same object is called whenever pathfinding is required, and it uses the preallocated location data it needs and ignores the rest. On its own, this implementation will be a little more complicated, and could be considerably more complex if multiple characters who need pathfinding must queue to use the one pathfinding object. To avoid complicating the underlying algorithm this book presents them in their simplest form: without regard for allocation.

Cache

Memory size alone isn't the only limitation on memory use. The time it takes to access memory from the RAM and prepare it for use by the processor is significantly longer than the time it takes for the processor to perform its operations. If processors had to rely on the main RAM, they'd be constantly stalled waiting for data.

All modern processors use at least one level of cache: a copy of the RAM held in the processor that can be very quickly manipulated. Cache is typically fetched in pages; a whole section of main memory is streamed to the processor. It can then be manipulated at will. When the processor has done its work, the cached memory is sent back to the main memory. The processor typically cannot work on the main memory: all the memory it needs must be on cache. An operating system may add additional complexity to this, as a memory request may have to pass through an operating system routine that translates the request into a request for real or virtual memory. This can introduce further constraints, as two bits of physical memory with a similar mapped address might not be available at the same time (called an *aliasing failure*).

Multiple levels of cache work the same way as a single cache. A large amount of memory is fetched to the lowest level cache, a subset of that is fetched to each higher level cache, and the processor only ever works on the highest level.

If an algorithm uses data spread around memory, then it is unlikely that the right memory will be in the cache from moment to moment. These cache misses are costly in time. The processor has to fetch a whole new chunk of memory into the cache for one or two instructions, then it has to stream it all back out and request another block. A good profiling system will show when cache misses are happening. In my experience, even in languages that don't give

you control over memory layout, dramatic speed ups can be achieved by making sure that all the data needed for one algorithm are kept in the same place, in the same few objects.

In this book, for ease of understanding, I've used an object-oriented style to lay out the data. All the data for a particular game object are kept together. This may not be the most cache-efficient solution. In a game with 1000 characters, it may be better to keep all their positions together in an array, so algorithms that make calculations based on location don't need to constantly jump around memory. As with all optimizations, profiling is everything, but a general level of efficiency can be gained by programming with data coherence in mind.

2.3.3 PLATFORMS

With the centralization of the industry around a few game engines, platform differences have less of an impact on AI design than they used to. Graphics programmers may still have to worry about console versus mobile, for example. But AI programming tends to be more general. In this section I will consider each of the major platforms for games, highlighting any issues specific to AI code.

PC

PCs can be the most powerful games machines, with hard-core gamers buying high-end expensive hardware. But they can be frustrating for developers because of their lack of consistency. Where a console has fixed hardware (or at least relatively few variations), there is a bewildering array of different configurations for PCs. There is a vast difference between a machine with a pair of top of the range video cards, SSD drives and fast memory, and a budget PC with integrated graphics.

Things are easier than they were: low-level developers rely on application programming interfaces (APIs) such as Vulkan and DirectX to insulate them from most hardware specifics, but the game still needs to detect feature support and speed and adjust accordingly. Developers working in an engine such as Unity and Unreal have it even easier, but may still need to use the built-in feature detection to ensure their game runs well on all systems.

Working with PCs involves building software that can scale from a casual gamer's limited system to the hard-core fan's up-to-date hardware. For graphics, this scaling can be reasonably modular; for example, for low-specification machines we switch off advanced rendering features. A simpler shadow algorithm might be used, or physically based shaders might be replaced by simple texture mapping. A change in graphics sophistication usually doesn't change gameplay.

AI is different. If the AI gets less time to work, how should it respond? It can try to perform less work. This is effectively the same as having more stupid AI, and can affect the difficulty level of the game. It is probably not acceptable to have your game be easier on lower specification machines. Similarly, if we try to perform the same amount of work, it might take longer. This can mean a lower frame rate, or it can mean more frames between characters

making decisions. Slow-to-react characters are also often easier to play against and can cause the same problems with QA.

The solution used by most developers is to target AI at the lowest common denominator: the minimum specification machine listed in the technical design document. The AI time doesn't scale at all with the capabilities of the machine. Faster machines simply use proportionally less of their processing budget on AI.

There are many games, however, where scalable AI is feasible. Many games use AI to control ambient characters: pedestrians walking along the sidewalk, members of the crowd cheering a race, or flocks of birds swarming in the sky. This kind of AI is freely scalable: more characters can be used when the processor time is available. [Chapter 10](#) covers some techniques for level of detail AI that can cope with this scalability.

Console

Consoles can be simpler to work with than a PC. You know exactly the machine you are targeting, and you can usually see code in operation on your target machine. There is no future proofing for new hardware or ever-changing versions of APIs to worry about.

Developers working with next-generation technology often don't have the exact specs of the final machine or a reliable hardware platform (initial development kits are often little more than a dedicated emulator), but most console development has a fairly fixed target.

The technical requirements checklist (TRC) process, by which a console manufacturer places minimum standards on the operation of a game, serves to fix things like frame rates (although different territories may vary—PAL and NTSC, for example). This means that AI budgets can be locked down in terms of a fixed number of milliseconds. In turn, this makes it much easier to work out what algorithms can be used and to have a fixed target for optimization (provided that the budget isn't slashed at the last milestone to make way for the latest graphics technique used in a competitor's game).

The same game engines used for PC development target consoles, making cross-platform development much easier than it has been in the past. Fortunately few AI developers creating games are now working with the low-level details of a particular console. Almost all the low-level code is handled by engines or middleware.

Mobile

Apple launched the iPhone in 2007, ushering in a revolution in gaming as big as anything since the home consoles of the 80s. When the first edition of this book was released, in 2006, mobile gaming consisted of dedicated handheld consoles like the PlayStation Portable (PSP) and Nintendo's GameBoy advance. Now almost 100% of the market is for phones and tablets.

There are two platforms in the space: Apple, with its iOS devices (iPhone, iPad, iPod Touch), and Android. Until recently these were very different, and required games to be coded for each individually. Although both can use low-level languages such as C and C++, for

higher-level languages Apple encourages the use of Swift (it formerly used Objective-C), and Android Java (or languages that compile into Java bytecode, such as Kotlin).

Both the major game engines by market share (Unreal and Unity), as well as many smaller competitors (e.g. Godot) support mobile platforms with the same game code, making platform-specific implementation unnecessary. There has been a big shift toward mobile developers working in a cross-platform way, using these tools. Factoring in the Steam platform as a viable marketplace for mobile games running on PC, I think there is no doubt this trend will soon become almost ubiquitous.

Smartphones capable of running games are powerful machines, comparable to last generation consoles, and PCs 5-10 old. There is no longer any practical difference between the kinds of AI that can be run on a PC or console and those that can be run on mobile. Phones may require simpler graphics or smaller crowd sizes, but in terms of algorithms, the same things now apply.

Virtual and Augmented Reality

At the time of writing, in early 2019, virtual and augmented reality are both extremely hyped, and a tiny proportion of the games market. The technology and the market is in rapid flux, and beyond generalities, very little that could be said now would be true in two years.

Virtual reality (VR) attempts to immerse the player in the game world by providing a stereoscopic 3D point of view. Depending on the hardware, a player's motion may also be detected and incorporated as movement in the game. VR requires separate views of the scene to be rendered for each eye, and to avoid motion sickness, higher frame rates are typically targeted (90fps, for example).

Up to this point most virtual reality devices have been displays tethered to an existing games machine, such as a PC (Oculus Rift and Vive), a console (PlayStation VR), or a phone (Gear VR). At the time of writing, companies are beginning to release standalone VR Products, based on mobile processors, with approximately similar performance to high-end phones.

Augmented reality (AR) uses semitransparent displays to add computer-generated elements to the real world. Although Microsoft released a development kit in early 2016, a consumer version has not yet followed. Magic Leap released their product in 2018, but saw limited demand. Augmented reality may also refer to games that use a mobile phone camera, and add computer-generated elements to the captured images. In that sense, *Pokémon Go* [150], for example, is considered an augmented reality game, but does not require specialist hardware.

While the visual presentation of VR games may be unconventional, the game logic rarely is. Most commercial game engines support VR, AR on mobile via camera, and are positioned to offer hardware AR support, when products are announced. VR and AR games are similar enough in design not to need unusual AI algorithms. It remains to be seen whether these platforms open new design possibilities. It also remains to be seen whether these platforms become a significant part of the industry.

2.4 THE AI ENGINE

When I started in the industry, a game was mostly built from scratch. Some bits of code were dragged from previous projects, and some bits were reworked and reused, but most were new. A handful of companies used the same basic code to write multiple games, as long as the games were a similar style and genre. LucasArts' SCUMM engine, for example, was a gradually evolving game engine used to power many point-and-click adventure games.

Since then, game engines have become ubiquitous, a consistent technical platform on which multiple games are built. Low-level routines (like talking to the operating system, loading textures, model file formats, and so on) are shared among all titles, a set of tools are available for a broad range of games (e.g. 2D graphics, 3D graphics, networking), and finally interfaces are provided to add game-specific code on top. Originally these engines belonged to individual companies, but over time only the very largest companies could afford to keep their engines up-to-date. It is now common for large developers to license commercial engines.

The way AI is developed has changed, also. Initially, the AI was written for each game and sometimes for each character. Now there is an increasing tendency to have general AI routines, either built into the game engine, available to license as commercial add-ons, or created and reused in-house by a developer. This allows individual characters to be designed by level editors, game designers, or technical artists. The engine structure is fixed, and the AI for each character combines the components in an appropriate way. The algorithms described in this book are exposed to non-specialists through graphical front-ends. Dragging boxes and lines allows anyone to create a finite state machine ([Section 5.3](#)) or behavior tree ([Section 5.4](#)), for example.

So, building a game engine involves building AI tools that can be easily reused, combined, and applied in interesting ways. To support this, we need an AI structure that makes sense over multiple genres.

2.4.1 STRUCTURE OF AN AI ENGINE

In my experience, there are a few basic facilities that need to be in place for a general AI system. They conform to the model of AI given in [Figure 2.1](#).

First, we must have some kind of infrastructure in two categories: a general mechanism for managing AI behaviors (deciding which behavior gets to run when, and so on) and a world interfacing system for getting information into the AI. Every AI algorithm needs to honor these mechanisms.

Second, we must have a means to turn whatever the AI wants to do into action on-screen. This consists of standard interfaces to a movement and an animation controller, which can turn requests such as “pull lever 1” or “walk stealthily to position x, y ” into action.

Third, a standard behavior structure must serve as a liaison between the two. It is almost guaranteed that you will need to write one or two AI algorithms for each new game. Having

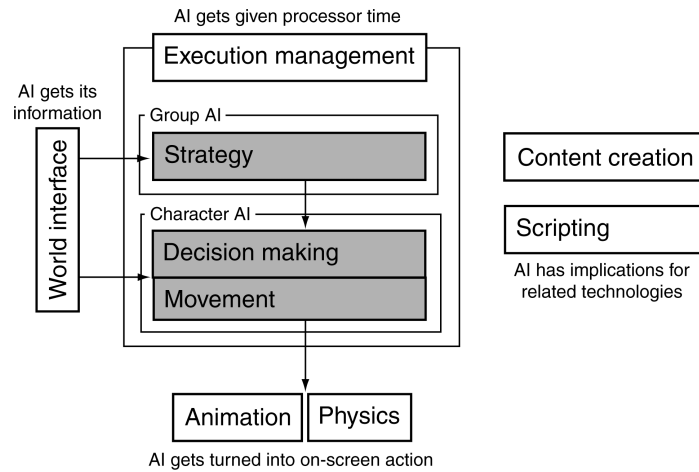


Figure 2.1: The AI model

all AI conform to the same structure helps this immensely. New code can be in development while the game is running, and the new AI can simply replace placeholder behaviors when it is ready.

All this needs to be thought out in advance. The structure needs to be in place before you get well into your AI coding. [Part III](#) of this book discusses support technologies, which are the first thing to implement in an AI engine. The individual techniques can then slot in.

Game engines do some of this for you, but not all. Each engine has its own mechanism to make sure your code is run, often in the form of base classes that you should derive from. But you may need to provide more fine-grained control: not every character needs its AI run in every frame. They also provide standard mechanisms for scheduling animations, but less often character movement. And they may provide basic tools for working out which character knows what (such as a built-in line of sight or sight cone check), but will need custom implementations for anything more complex. Unless you intend to use very simple techniques, you will need to create some infrastructure, and possibly some tools in the editor to support it.

I'm not going to emphasize this structure throughout the book. There are techniques that I will cover that can work on their own, and all the algorithms are fairly independent. For a demo, or a simple game, it might be sufficient to just use the technique. But a good AI structure helps promote reuse, and reduces debugging and development time.

2.4.2 TOOL CONCERNS

The complete AI engine will have a central pool of AI algorithms that can be applied to many characters. The definition for a particular character's AI will therefore consist of data (which may include scripts in some scripting language), rather than compiled code. The data specifies how a character is put together: what techniques will be used and how those techniques are parameterized and combined.

This data needs to come from somewhere. Data can be manually created, but this is no better than writing the AI by hand each time. Flexible tools ensure that the artists and designers can create the content in an easy way, while allowing the content to be inserted into the game without manual help. These are usually created as custom modes in the game engine editor: a tool for setting up character decision making rules, or an overlay onto the level used to mark tactical locations or places to avoid.

The need to expose search tools has its own effect on the choice of AI techniques. It is easy to set up behaviors that always act the same way. Steering behaviors (covered in [Chapter 3](#)) are a good example: they tend to be very simple, they are easily parameterized (with the physical capabilities of a character), and they do not change from character to character.

It is more difficult to use behaviors that have lots of conditions, where the character needs to evaluate special cases. Many of those covered in [Chapter 5](#) operate this way. Those based on a tree (decision trees, behavior trees) are easier to represent visually. A rule-based system, on the other hand, needs to have complicated matching rules defined. When these are supported in a tool they typically look like program code, because a programming language is the most natural way to express them.

2.4.3 PUTTING IT ALL TOGETHER

The final structure of the AI engine might look something like [Figure 2.2](#). Data are created in a tool (the modeling or level editor), which is then packaged for use in the game. When a level is loaded, the game AI behaviors are created from level data and registered with the AI engine. During gameplay, the main game code calls the AI engine which updates the behaviors, getting information from the world interface and finally applying their output to the game data.

The particular techniques used depend heavily on the genre of the game being developed. We'll see a wide range of techniques for many different genres in this book. As you develop your game AI, you'll need to take a mix and match approach to get the behaviors you are looking for. The final part of the book gives some hints on this; it looks at how the AI for games in the major genres are put together piece by piece.

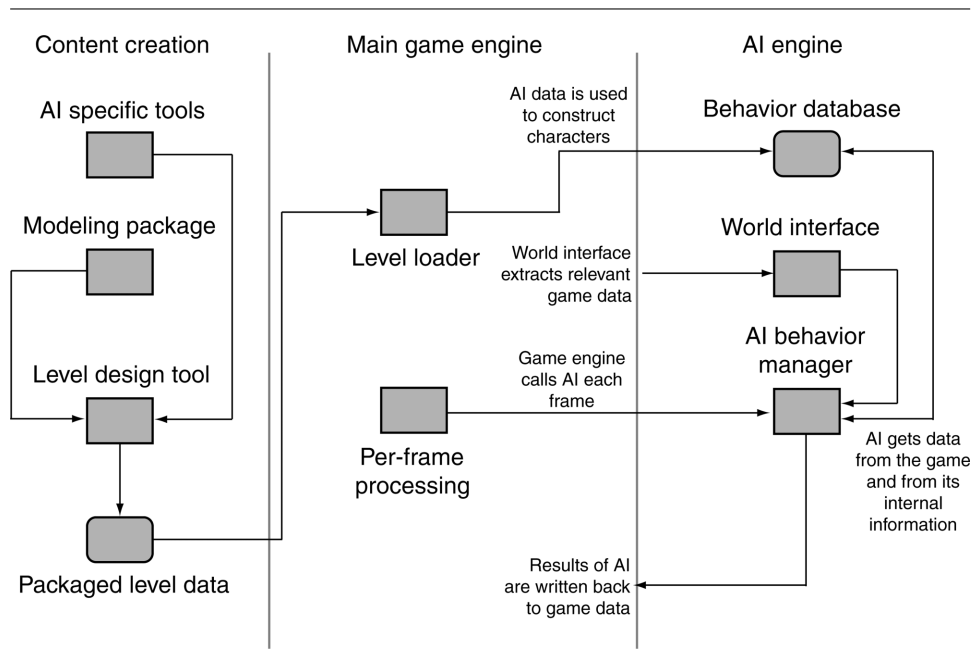


Figure 2.2: AI schematic

PART II

Techniques



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>



MOVEMENT

ONE of the most fundamental requirements of game AI is to sensibly move characters around. Even the earliest AI-controlled characters (the ghosts in *Pac-Man*, for example, or the opposing bat in some *Pong* variants) had movement algorithms that weren't far removed from modern games.

Movement forms the lowest level of AI techniques in our model, shown in [Figure 3.1](#).

Many games, including some with quite decent-looking AI, rely solely on movement algorithms and don't have any more advanced decision making. At the other extreme, some games don't need moving characters at all. Resource management games and turn-based games often don't need movement algorithms; once a decision is made where to move, the character can simply be placed there.

There is also some degree of overlap between AI and animation; animation is also about movement. This chapter looks at large-scale movement: the movement of characters around the game level, rather than the movement of their limbs or faces. The dividing line isn't always clear, however. In many games, animation can take control over a character, including some large-scale movement. This may be as simple as the character moving a few steps to pull a lever. Or as complex as mini cut-scenes, completely animated, that seamlessly transition into and out of gameplay. These are not AI driven and therefore aren't covered here.

This chapter will look at a range of different AI-controlled movement algorithms, from the simple *Pac-Man* level up to the complex steering behaviors used for driving a racing car or piloting a spaceship in three dimensions.

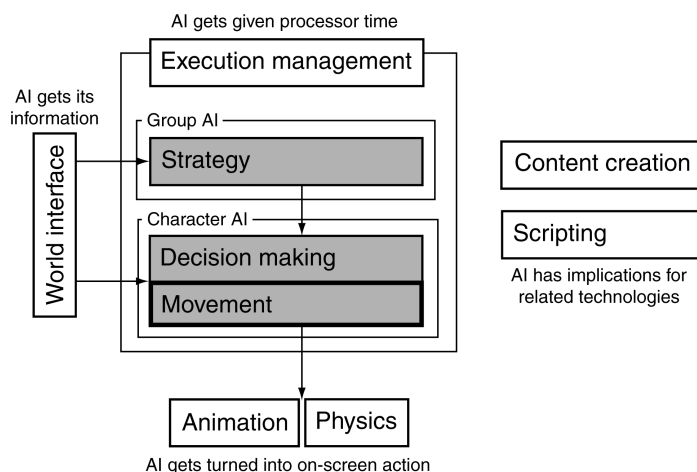


Figure 3.1: The AI model

3.1 THE BASICS OF MOVEMENT ALGORITHMS

Unless you're writing an economic simulator, chances are the characters in your game need to move around. Each character has a current position and possibly additional physical properties that control its movement. A movement algorithm is designed to use these properties to work out where the character should be next.

All movement algorithms have this same basic form. They take geometric data about their own state and the state of the world, and they come up with a geometric output representing the movement they would like to make. [Figure 3.2](#) shows this schematically. In the figure, the velocity of a character is shown as optional because it is only needed for certain classes of movement algorithms.

Some movement algorithms require very little input: just the position of the character and the position of an enemy to chase, for example. Others require a lot of interaction with the game state and the level geometry. A movement algorithm that avoids bumping into walls, for example, needs to have access to the geometry of the wall to check for potential collisions.

The output can vary too. In most games it is normal to have movement algorithms output a desired velocity. A character might see its enemy to the west, for example, and respond that its movement should be westward at full speed. Often, characters in older games only had two speeds: stationary and running (with maybe a walk speed in there, too, for patrolling). So the output was simply a direction to move in. This is kinematic movement; it does not account for how characters accelerate and slow down.

It is common now for more physical properties to be taken into account. Producing movement algorithms I will call "steering behaviors." Steering behaviors is the name given by Craig

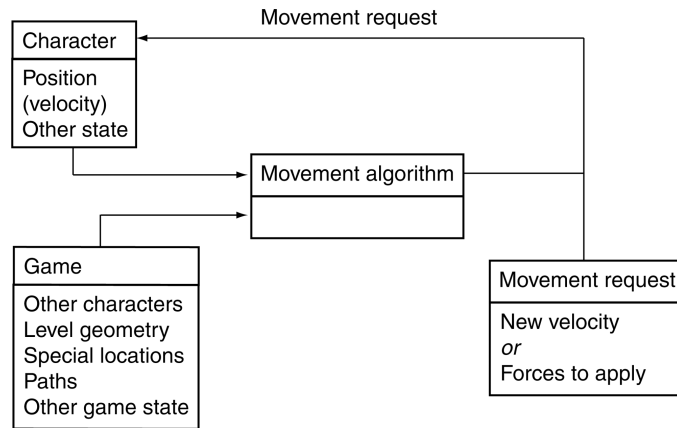


Figure 3.2: The movement algorithm structure

Reynolds to his movement algorithms [51]; they are not kinematic, but dynamic. Dynamic movement takes account of the current motion of the character. A dynamic algorithm typically needs to know the current velocities of the character as well as its position. A dynamic algorithm outputs forces or accelerations with the aim of changing the velocity of the character.

Dynamics adds an extra layer of complexity. Let's say your character needs to move from one place to another. A kinematic algorithm simply gives the direction to the target; your character moves in that direction until it arrives, whereupon the algorithm returns no direction. A dynamic movement algorithm needs to work harder. It first needs to accelerate in the right direction, and then as it gets near its target it needs to accelerate in the opposite direction, so its speed decreases at precisely the correct rate to slow it to a stop at exactly the right place. Because Craig's work is so well known, in the rest of this chapter I'll usually follow the most common terminology and refer to all dynamic movement algorithms as steering behaviors.

Craig Reynolds also invented the flocking algorithm used in countless films and games to animate flocks of birds or herds of other animals. We'll look at this algorithm later in the chapter. Because flocking is the most famous steering behavior, all steering algorithms (in fact, all movement algorithms) are sometimes wrongly called "flocking."

3.1.1 TWO-DIMENSIONAL MOVEMENT

Many games have AI that works in two dimensions (2D). Even games that are rendered in three dimensions (3D), usually have characters that are under the influence of gravity, sticking them to the floor and constraining their movement to two dimensions.

A lot of movement AI can be achieved in 2D, and most of the classic algorithms are only

defined for this case. Before looking at the algorithms themselves, we need to quickly cover the data needed to handle 2D math and movement.

Characters as Points

Although a character usually consists of a 3D model that occupies some space in the game world, many movement algorithms assume that the character can be treated as a single point. Collision detection, obstacle avoidance, and some other algorithms use the size of the character to influence their results, but movement itself assumes the character is at a single point.

This is a process similar to that used by physics programmers who treat objects in the game as a “rigid body” located at its center of mass. Collision detection and other forces can be applied anywhere on the object, but the algorithm that determines the movement of the object converts them so it can deal only with the center of mass.

3.1.2 STATICS

Characters in two dimensions have two linear coordinates representing the position of the object. These coordinates are relative to two world axes that lie perpendicular to the direction of gravity and perpendicular to each other. This set of reference axes is termed the *orthonormal basis of the 2D space*.

In most games the geometry is typically stored and rendered in three dimensions. The geometry of the model has a 3D orthonormal basis containing three axes: normally called x , y , and z . It is most common for the y -axis to be in the opposite direction to gravity (i.e., “up”) and for the x and z axes to lie in the plane of the ground. Movement of characters in the game takes place along the x and z axes used for rendering, as shown in [Figure 3.3](#). For this reason this chapter will use the x and z axes when representing movement in two dimensions, even though books dedicated to 2D geometry tend to use x and y for the axis names.

In addition to the two linear coordinates, an object facing in any direction has one orientation value. The orientation value represents an angle from a reference axis. In our case we use a counterclockwise angle, in radians, from the positive z -axis. This is fairly standard in game engines; by default (i.e., with zero orientation a character is looking down the z -axis).

With these three values the static state of a character can be given in the level, as shown in [Figure 3.4](#).

Algorithms or equations that manipulate this data are called *static* because the data do not contain any information about the movement of a character.

We can use a data structure of the form:

```
1 class Static:
2     position: Vector
3     orientation: float
```

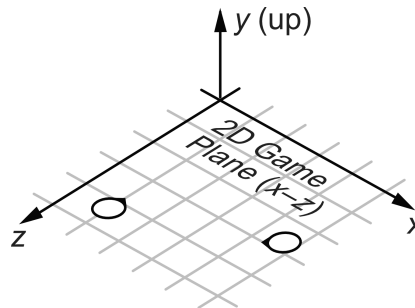


Figure 3.3: The 2D movement axes and the 3D basis

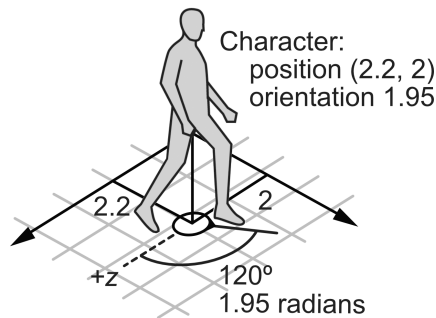


Figure 3.4: The positions of characters in the level

I will use the term *orientation* throughout this chapter to mean the direction in which a character is facing. When it comes to rendering characters, we will make them appear to face one direction by rotating them (using a rotation matrix). Because of this, some developers refer to orientation as *rotation*. I will use rotation in this chapter only to mean the process of changing orientation; it is an active process.

2½ Dimensions

Some of the math involved in 3D geometry is complicated. The linear movement in three dimensions is quite simple and a natural extension of 2D movement, but representing an orientation has tricky consequences that are better to avoid (at least until the end of the chapter).

As a compromise, developers often use a hybrid of 2D and 3D calculations which is known as 2½D, or sometimes four degrees of freedom.

In 2½ dimensions we deal with a full 3D position but represent orientation as a single value, as if we are in two dimensions. This is quite logical when you consider that most games involve characters under the influence of gravity. Most of the time a character's third dimension is constrained because it is pulled to the ground. In contact with the ground, it is effectively operating in two dimensions, although jumping, dropping off ledges, and using elevators all involve movement through the third dimension.

Even when moving up and down, characters usually remain upright. There may be a slight tilt forward while walking or running or a lean sideways out from a wall, but this tilting doesn't affect the movement of the character; it is primarily an animation effect. If a character remains upright, then the only component of its orientation we need to worry about is the rotation about the up direction.

This is precisely the situation we take advantage of when we work in 2½D, and the simplification in the math is worth the decreased flexibility in most cases.

Of course, if you are writing a flight simulator or a space shooter, then all the orientations are very important to the AI, so you'll have to support the mathematics for three-dimensional orientation. At the other end of the scale, if your game world is completely flat and characters can't jump or move vertically in any other way, then a strict 2D model is needed. In the vast majority of cases, 2½D is an optimal solution. I'll cover full 3D motion at the end of the chapter, but aside from that, all the algorithms described in this chapter are designed to work in 2½D.

Math

In the remainder of this chapter I will assume that you are comfortable using basic vector and matrix mathematics (i.e., addition and subtraction of vectors, multiplication by a scalar). Explanations of vector and matrix mathematics, and their use in computer graphics, are beyond the scope of this book. If you can find it, the older book Schneider and Eberly [56], covers mathematical topics in computer games to a much deeper level. An excellent, and more recent alternative is [36].

Positions are represented as a vector with x and z components of position. In 2½D, a y component is also given.

In two dimensions we need only an angle to represent orientation. The angle is measured from the positive z -axis, in a right-handed direction about the positive y -axis (counterclockwise as you look down on the x - z plane from above). [Figure 3.4](#) gives an example of how the scalar orientation is measured.

It is more convenient in many circumstances to use a vector representation of orientation. In this case the vector is a unit vector (it has a length of one) in the direction that the character is facing. This can be directly calculated from the scalar orientation using simple trigonometry:

$$\vec{\omega}_v = \begin{bmatrix} \sin \omega_s \\ \cos \omega_s \end{bmatrix}$$

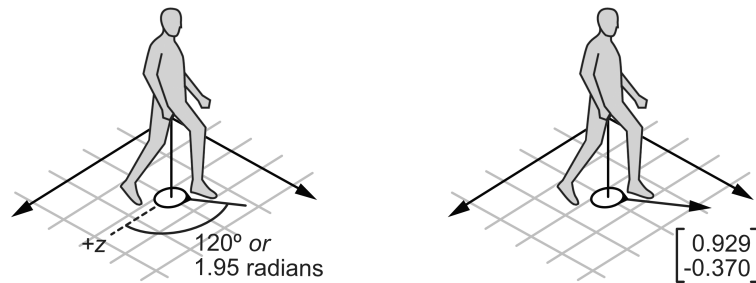


Figure 3.5: The vector form of orientation

where ω_s is the orientation as a scalar (i.e. a single number representing the angle), and $\vec{\omega}_v$ is the orientation expressed as a vector. I am assuming a right-handed coordinate system here, in common with most of the game engines I've worked on,¹ if you use a left-handed system, then simply flip the sign of the x coordinate:

$$\vec{\omega}_v = \begin{bmatrix} -\sin \omega_s \\ \cos \omega_s \end{bmatrix}$$

If you draw the vector form of the orientation, it will be a unit length vector in the direction that the character is facing, as shown in Figure 3.5.

3.1.3 KINEMATICS

So far, each character has two associated pieces of information: its position and its orientation. We can create movement algorithms to calculate a target velocity based on position and orientation alone, allowing the output velocity to change instantly.

While this is fine for many games, it can look unrealistic. A consequence of Newton's laws of motion is that velocities cannot change instantly in the real world. If a character is moving in one direction and then instantly changes direction or speed, it may look odd. To make smooth motion or to cope with characters that can't accelerate very quickly, we need either to use some kind of smoothing algorithm or to take account of the current velocity and use accelerations to change it.

To support this, the character keeps track of its current velocity as well as position. Algorithms can then operate to change the velocity slightly at each frame, giving a smooth motion.

Characters need to keep track of both their linear and their angular velocities. Linear velocity has both x and z components, the speed of the character in each of the axes in the

1. Left-handed coordinates work just as well with all the algorithms in this chapter. See [13] for more details of the difference, and how to convert between them.

orthonormal basis. If we are working in 2½D, then there will be three linear velocity components, in x , y , and z .

The angular velocity represents how fast the character's orientation is changing. This is given by a single value: the number of radians per second that the orientation is changing.

I will call angular velocity *rotation*, since rotation suggests motion. Linear velocity will normally be referred to as simply velocity. We can therefore represent all the kinematic data for a character (i.e., its movement and position) in one structure:

```
1 class Kinematic:
2     position: Vector
3     orientation: float
4     velocity: Vector
5     rotation: float
```

Steering behaviors operate with these kinematic data. They return accelerations that will change the velocities of a character in order to move them around the level. Their output is a set of accelerations:

```
1 class SteeringOutput:
2     linear: Vector
3     angular: float
```

Independent Facing

Notice that there is nothing to connect the direction that a character is moving and the direction it is facing. A character can be oriented along the x -axis but be traveling directly along the z -axis. Most game characters should not behave in this way; they should orient themselves so they move in the direction they are facing.

Many steering behaviors ignore facing altogether. They operate directly on the linear components of the character's data. In these cases the orientation should be updated so that it matches the direction of motion.

This can be achieved by directly setting the orientation to the direction of motion, but doing so may mean the orientation changes abruptly.

A better solution is to move it a proportion of the way toward the desired direction: to smooth the motion over many frames. In [Figure 3.6](#), the character changes its orientation to be halfway toward its current direction of motion in each frame. The triangle indicates the orientation, and the gray shadows show where the character was in previous frames, to illustrate its motion.

Updating Position and Orientation

If your game uses a physics engine, it may be used to update the position and orientation of characters. Even with a physics engine, some developers prefer to use only its collision

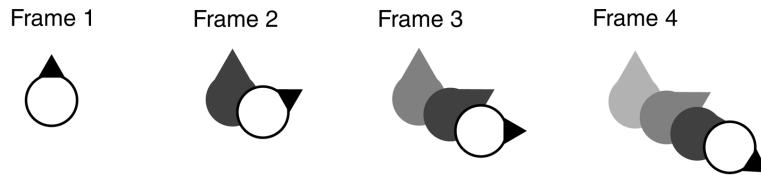


Figure 3.6: Smoothing facing direction of motion over multiple frames

detection on characters, and code custom movement controllers. So if you need to update position and orientation manually, you can use a simple algorithm of the form:

```

1 class Kinematic:
2     # ... Member data as before ...
3
4     function update(steering: SteeringOutput, time: float):
5         # Update the position and orientation.
6         half_t_sq: float = 0.5 * time * time
7         position += velocity * time + steering.linear * half_t_sq
8         orientation += rotation * time + steering.angular * half_t_sq
9
10        # and the velocity and rotation.
11        velocity += steering.linear * time
12        rotation += steering.angular * time

```

The updates use high-school physics equations for motion. If the frame rate is high, then the update time passed to this function is likely to be very small. The square of this time is likely to be even smaller, and so the contribution of acceleration to position and orientation will be tiny. It is more common to see these terms removed from the update algorithm, to give what's known as the Newton-Euler-1 integration update:

```

1 class Kinematic:
2     # ... Member data as before ...
3
4     function update(steering: SteeringOutput, time: float):
5         # Update the position and orientation.
6         position += velocity * time
7         orientation += rotation * time
8
9         # and the velocity and rotation.
10        velocity += steering.linear * time
11        rotation += steering.angular * time

```

This is the most common update used for games. Note that, in both blocks of code I've assumed that we can do normal mathematical operations with vectors, such as addition and

multiplication by a scalar. Depending on the language you are using, you may have to replace these primitive operations with function calls.

The *Game Physics* [12] book in the Morgan Kaufmann Interactive 3D Technology series, and my *Game Physics Engine Development* [40], also in that series, have a complete analysis of different update methods and cover a more complete range of physics tools for games (as well as detailed implementations of vector and matrix operations).

Variable Frame Rates

So far I have assumed that velocities are given in units per second rather than per frame. Older games often used per-frame velocities, but that practice largely died out, before being resurrected occasionally when gameplay began to be processed in a separate execution thread with a fixed frame rate, separate to graphics updates (Unity's `FixedUpdate` function compared to its variable `Update`, for example). Even with these functions available, it is more flexible to support variable frame rates, in case the fixed update rate needs changing. For this reason I will continue to use an explicit update time.

If the character is known to be moving at 1 meter per second and the last frame was of 20 milliseconds' duration, then they will need to move 20 millimeters.

Forces and Actuation

In the real world we can't simply apply an acceleration to an object and have it move. We apply forces, and the forces cause a change in the kinetic energy of the object. They will accelerate, of course, but the acceleration will depend on the inertia of the object. The inertia acts to resist the acceleration; with higher inertia, there is less acceleration for the same force.

To model this in a game, we could use the object's mass for the linear inertia and the moment of inertia (or inertia tensor in three dimensions) for angular acceleration.

We could continue to extend the character data to keep track of these values and use a more complex update procedure to calculate the new velocities and positions. This is the method used by physics engines: the AI controls the motion of a character by applying forces to it. These forces represent the ways in which the character can affect its motion. Although not common for human characters, this approach is almost universal for controlling cars in driving games: the drive force of the engine and the forces associated with the steering wheels are the only ways in which the AI can control the movement of the car.

Because most well-established steering algorithms are defined with acceleration outputs, it is not common to use algorithms that work directly with forces. Usually, the movement controller considers the dynamics of the character in a post-processing step called *actuation*.

Actuation takes as input a desired change in velocity, the kind that would be directly applied in a kinematic system. The actuator then calculates the combination of forces that it can apply to get as near as possible to the desired velocity change.

At the simplest level this is just a matter of multiplying the acceleration by the inertia to

give a force. This assumes that the character is capable of applying any force, however, which isn't always the case (a stationary car can't accelerate sideways, for example). Actuation is a major topic in AI and physics integration, and I'll return to actuation at some length in [Section 3.8](#) of this chapter.

3.2 KINEMATIC MOVEMENT ALGORITHMS

Kinematic movement algorithms use static data (position and orientation, no velocities) and output a desired velocity. The output is often simply an on or off and a target direction, moving at full speed or being stationary. Kinematic algorithms do not use acceleration, although the abrupt changes in velocity might be smoothed over several frames.

Many games simplify things even further and force the orientation of a character to be in the direction it is traveling. If the character is stationary, it faces either a pre-set direction or the last direction it was moving in. If its movement algorithm returns a target velocity, then that is used to set its orientation.

This can be done simply with the function:

```

1 function newOrientation(current: float, velocity: Vector) -> float:
2     # Make sure we have a velocity.
3     if velocity.length() > 0:
4         # Calculate orientation from the velocity.
5         return atan2(-static.x, static.z)
6
7     # Otherwise use the current orientation.
8     else:
9         return current

```

We'll look at two kinematic movement algorithms: seeking (with several of its variants) and wandering. Building kinematic movement algorithms is extremely simple, so we'll only look at these two as representative samples before moving on to dynamic movement algorithms, the bulk of this chapter.

I can't stress enough, however, that this brevity is not because they are uncommon or unimportant. Kinematic movement algorithms still form the bread and butter of movement systems in a lot of games. The dynamic algorithms in the rest of the book are widespread, where character movement is controlled by a physics engine, but they are still not ubiquitous.

3.2.1 SEEK

A kinematic seek behavior takes as input the character's static data and its target's static data. It calculates the direction from the character to the target and requests a velocity along this line. The orientation values are typically ignored, although we can use the `newOrientation` function above to face in the direction we are moving.

The algorithm can be implemented in a few lines:

```

1  class KinematicSeek:
2      character: Static
3      target: Static
4
5      maxSpeed: float
6
7      function getSteering() -> KinematicSteeringOutput:
8          result = new KinematicSteeringOutput()
9
10         # Get the direction to the target.
11         result.velocity = target.position - character.position
12
13         # The velocity is along this direction, at full speed.
14         result.velocity.normalize()
15         result.velocity *= maxSpeed
16
17         # Face in the direction we want to move.
18         character.orientation = newOrientation(
19             character.orientation,
20             result.velocity)
21
22         result.rotation = 0
23         return result

```

where the `normalize` method applies to a vector and makes sure it has a length of one. If the vector is a zero vector, then it is left unchanged.

Data Structures and Interfaces

We use the `Static` data structure as defined at the start of the chapter and a `KinematicSteeringOutput` structure for output. The `KinematicSteeringOutput` structure has the following form:

```

1  class KinematicSteeringOutput:
2      velocity: Vector
3      rotation: float

```

In this algorithm rotation is never used; the character's orientation is simply set based on their movement. You could remove the call to `newOrientation` if you want to control orientation independently somehow (to have the character aim at a target while moving, as in *Tomb Raider* [95], or twin-stick shooters such as *The Binding of Isaac* [151], for example).

Performance

The algorithm is $O(1)$ in both time and memory.

Flee

If we want the character to run away from the target, we can simply reverse the second line of the `getSteering` method to give:

```
1 # Get the direction away from the target.
2 steering.velocity = character.position - target.position
```

The character will then move at maximum velocity in the opposite direction.

Arriving

The algorithm above is intended for use by a chasing character; it will never reach its goal, but it continues to seek. If the character is moving to a particular point in the game world, then this algorithm may cause problems. Because it always moves at full speed, it is likely to overshoot an exact spot and wiggle backward and forward on successive frames trying to get there. This characteristic wiggle looks unacceptable. We need to end by standing stationary at the target spot.

To achieve this we have two choices. We can just give the algorithm a large radius of satisfaction and have it be satisfied if it gets closer to its target than that. Alternatively, if we support a range of movement speeds, then we could slow the character down as it reaches its target, making it less likely to overshoot.

The second approach can still cause the characteristic wiggle, so we benefit from blending both approaches. Having the character slow down allows us to use a much smaller radius of satisfaction without getting wiggle and without the character appearing to stop instantly.

We can modify the seek algorithm to check if the character is within the radius. If so, it doesn't worry about outputting anything. If it is not, then it tries to reach its target in a fixed length of time. (I've used a quarter of a second, which is a reasonable figure; you can tweak the value if you need to.) If this would mean moving faster than its maximum speed, then it moves at its maximum speed. The fixed time to target is a simple trick that makes the character slow down as it reaches its target. At 1 unit of distance away it wants to travel at 4 units per second. At a quarter of a unit of distance away it wants to travel at 1 unit per second, and so on. The fixed length of time can be adjusted to get the right effect. Higher values give a more gentle deceleration, and lower values make the braking more abrupt.

The algorithm now looks like the following:

```
1 class KinematicArrive:
2     character: Static
3     target: Static
```

```

4
5     maxSpeed: float
6
7     # The satisfaction radius.
8     radius: float
9
10    # The time to target constant.
11    timeToTarget: float = 0.25
12
13    function getSteering() -> KinematicSteeringOutput:
14        result = new KinematicSteeringOutput()
15
16        # Get the direction to the target.
17        result.velocity = target.position - character.position
18
19        # Check if we're within radius.
20        if result.velocity.length() < radius:
21            # Request no steering.
22            return null
23
24        # We need to move to our target, we'd like to
25        # get there in timeToTarget seconds.
26        result.velocity /= timeToTarget
27
28        # If this is too fast, clip it to the max speed.
29        if result.velocity.length() > maxSpeed:
30            result.velocity.normalize()
31            result.velocity *= maxSpeed
32
33        # Face in the direction we want to move.
34        character.orientation = newOrientation(
35            character.orientation,
36            result.velocity)
37
38        result.rotation = 0
39        return result

```

I've assumed a length function that gets the length of a vector.

3.2.2 WANDERING

A kinematic wander behavior always moves in the direction of the character's current orientation with maximum speed. The steering behavior modifies the character's orientation, which allows the character to meander as it moves forward. [Figure 3.7](#) illustrates this. The character is shown at successive frames. Note that it moves only forward at each frame (i.e., in the direction it was facing at the previous frame).

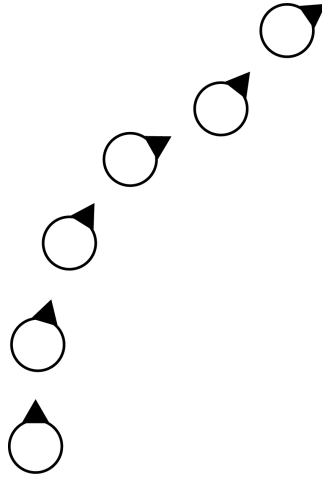


Figure 3.7: A character using kinematic wander

Pseudo-Code

It can be implemented as follows:

```
1 class KinematicWander:
2     character: Static
3     maxSpeed: float
4
5     # The maximum rotation speed we'd like, probably should be smaller
6     # than the maximum possible, for a leisurely change in direction.
7     maxRotation: float
8
9     function getSteering() -> KinematicSteeringOutput:
10         result = new KinematicSteeringOutput()
11
12         # Get velocity from the vector form of the orientation.
13         result.velocity = maxSpeed * character.orientation.asVector()
14
15         # Change our orientation randomly.
16         result.rotation = randomBinomial() * maxRotation
17
18         return result
```

Data Structures

Orientation values have been given an `asVector` function that converts the orientation into a direction vector using the formulae given at the start of the chapter.

Implementation Notes

I've used `randomBinomial` to generate the output rotation. This is a handy random number function that isn't common in the standard libraries of programming languages. It returns a random number between -1 and 1 , where values around zero are more likely. It can be simply created as:

```
1 function randomBinomial() -> float:  
2   return random() - random()
```

where `random()` returns a random number from 0 to 1.

For our wander behavior, this means that the character is most likely to keep moving in its current direction. Rapid changes of direction are less likely, but still possible.

3.3 STEERING BEHAVIORS

Steering behaviors extend the movement algorithms in the previous section by adding velocity and rotation. In some genres (such as driving games) they are dominant; in other genres they are more context dependent: some characters need them, some characters don't.

There is a whole range of different steering behaviors, often with confusing and conflicting names. As the field has developed, no clear naming schemes have emerged to tell the difference between one atomic steering behavior and a compound behavior combining several of them together.

In this book I'll separate the two: fundamental behaviors and behaviors that can be built up from combinations of these.

There are a large number of named steering behaviors in various papers and code samples. Many of these are variations of one or two themes. Rather than catalog a zoo of suggested behaviors, we'll look at the basic structures common to many of them before looking at some exceptions with unusual features.

3.3.1 STEERING BASICS

By and large, most steering behaviors have a similar structure. They take as input the kinematic of the character that is moving and a limited amount of target information. The target information depends on the application. For chasing or evading behaviors, the target is often another moving character. Obstacle avoidance behaviors take a representation of the collision

geometry of the world. It is also possible to specify a path as the target for a path following behavior.

The set of inputs to a steering behavior isn't always available in an AI-friendly format. Collision avoidance behaviors, in particular, need to have access to collision information in the level. This can be an expensive process: checking the anticipated motion of the character using ray casts or trial movement through the level.

Many steering behaviors operate on a group of targets. The famous flocking behavior, for example, relies on being able to move toward the average position of the flock. In these behaviors some processing is needed to summarize the set of targets into something that the behavior can react to. This may involve averaging properties of the whole set (to find and aim for their center of mass, for example), or it may involve ordering or searching among them (such as moving away from the nearest predator or avoiding bumping into other characters that are on a collision course).

Notice that the steering behavior isn't trying to do everything. There is no behavior to avoid obstacles while chasing a character and making detours via nearby power-ups. Each algorithm does a single thing and only takes the input needed to do that. To get more complicated behaviors, we will use algorithms to combine the steering behaviors and make them work together.

3.3.2 VARIABLE MATCHING

The simplest family of steering behaviors operates by variable matching: they try to match one or more of the elements of the character's kinematic to a single target kinematic.

We might try to match the position of the target, for example, not caring about the other elements. This would involve accelerating toward the target position and decelerating once we are near. Alternatively, we could try to match the orientation of the target, rotating so that we align with it. We could even try to match the velocity of the target, following it on a parallel path and copying its movements but staying a fixed distance away.

Variable matching behaviors take two kinematics as input: the character kinematic and the target kinematic. Different named steering behaviors try to match a different combination of elements, as well as adding additional properties that control how the matching is performed.

It is possible, but not particularly helpful, to create a general steering behavior capable of matching any subset of variables and simply tell it which combination of elements to match. I've made the mistake of trying this implementation myself. The problem arises when more than one element of the kinematic is being matched at the same time. They can easily conflict. We can match a target's position and orientation independently. But what about position and velocity? If we are matching velocities, between a character and its target, then we can't simultaneously be trying to get any closer.

A better technique is to have individual matching algorithms for each element and then combine them in the right combination later. This allows us to use any of the steering behavior

combination techniques in this chapter, rather than having one hard-coded. The algorithms for combining steering behaviors are designed to resolve conflicts and so are perfect for this task.

For each matching steering behavior, there is an opposite behavior that tries to get as far away from matching as possible. A behavior that tries to catch its target has an opposite that tries to avoid its target; a behavior that tries to avoid colliding with walls has an opposite that hugs them (perhaps for use in the character of a timid mouse) and so on. As we saw in the kinematic seek behavior, the opposite form is usually a simple tweak to the basic behavior. We will look at several steering behaviors in a pair with their opposite, rather than separating them into separate sections.

3.3.3 SEEK AND FLEE

Seek tries to match the position of the character with the position of the target. Exactly as for the kinematic seek algorithm, it finds the direction to the target and heads toward it as fast as possible. Because the steering output is now an acceleration, it will accelerate as much as possible.

Obviously, if it keeps on accelerating, its speed will grow larger and larger. Most characters have a maximum speed they can travel; they can't accelerate indefinitely. The maximum can be explicit, held in a variable or constant, or it might be a function of speed-dependent drag, slowing the character down more strongly the faster it goes.

With an explicit maximum, the current speed of the character (the length of the velocity vector) is checked regularly, and is trimmed back if it exceeds the maximum speed. This is normally done as a post-processing step of the update function. It is not normally performed in a steering behavior. For example,

```

1  class Kinematic:
2      # ... Member data as before ...
3
4      function update(steering: SteeringOutput,
5                      maxSpeed: float,
6                      time: float):
7          # Update the position and orientation.
8          position += velocity * time
9          orientation += rotation * time
10
11         # and the velocity and rotation.
12         velocity += steering.linear * time
13         rotation += steering.angular * time
14
15         # Check for speeding and clip.
16         if velocity.length() > maxSpeed:
17             velocity.normalize()
18             velocity *= maxSpeed

```

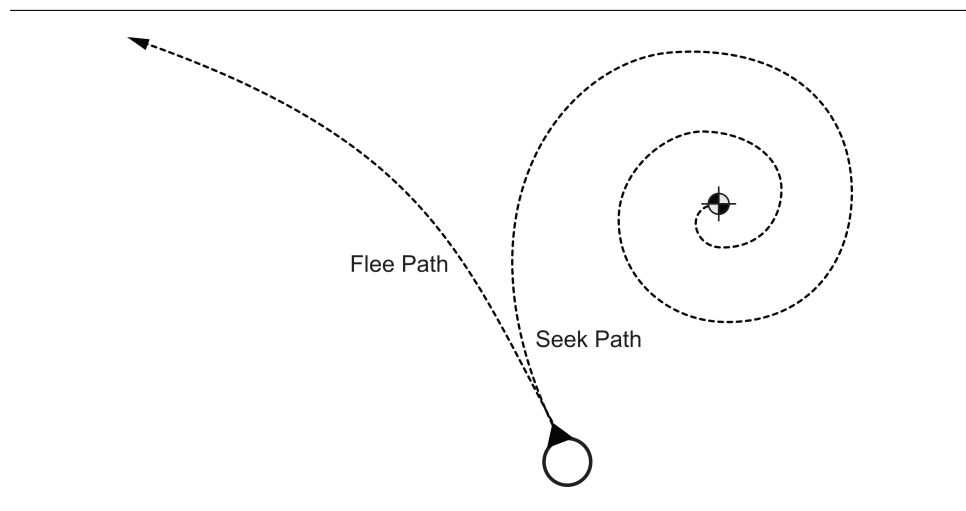


Figure 3.8: Seek and flee

Games that rely on physics engines typically include drag instead of a maximum speed (though they may use a maximum speed as well for safety). Without a maximum speed, the update does not need to check and clip the current velocity; the drag (applied in the physics update function) automatically limits the top speed.

Drag also helps another problem with this algorithm. Because the acceleration is always directed toward the target, if the target is moving, the seek behavior will end up orbiting rather than moving directly toward it. If there is drag in the system, then the orbit will become an inward spiral. If drag is sufficiently large, the player will not notice the spiral and will see the character simply move directly to its target.

[Figure 3.8](#) illustrates the path that results from the seek behavior and its opposite, the flee path, described below.

Pseudo-Code

The dynamic seek implementation looks very similar to our kinematic version:

```

1 class Seek:
2     character: Kinematic
3     target: Kinematic
4
5     maxAcceleration: float
6
7     function getSteering() -> SteeringOutput:
8         result = new SteeringOutput()
```

```

9
10     # Get the direction to the target.
11     result.linear = target.position - character.position
12
13     # Give full acceleration along this direction.
14     result.linear.normalize()
15     result.linear *= maxAcceleration
16
17     result.angular = 0
18     return result

```

Note that I've removed the change in orientation that was included in the kinematic version. We can simply set the orientation, as we did before, but a more flexible approach is to use variable matching to make the character face in the correct direction. The align behavior, described below, gives us the tools to change orientation using angular acceleration. The “look where you're going” behavior uses this to face the direction of movement.

Data Structures and Interfaces

This class uses the `SteeringOutput` structure we defined earlier in the chapter. It holds linear and angular acceleration outputs.

Performance

The algorithm is again $O(1)$ in both time and memory.

Flee

Flee is the opposite of seek. It tries to get as far from the target as possible. Just as for kinematic flee, we simply need to flip the order of terms in the second line of the function:

```

1  # Get the direction to the target.
2  steering.linear = character.position - target.position

```

The character will now move in the opposite direction of the target, accelerating as fast as possible.

3.3.4 ARRIVE

Seek will always move toward its goal with the greatest possible acceleration. This is fine if the target is constantly moving and the character needs to give chase at full speed. If the character

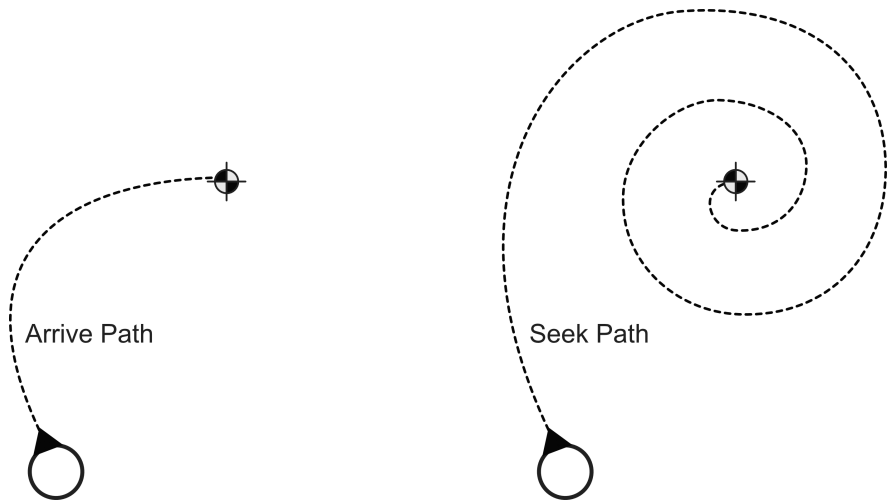


Figure 3.9: Seeking and arriving

arrives at the target, it will overshoot, reverse, and oscillate through the target, or it will more likely orbit around the target without getting closer.

If the character is supposed to arrive at the target, it needs to slow down so that it arrives exactly at the right location, just as we saw in the kinematic arrive algorithm. [Figure 3.9](#) shows the behavior of each for a fixed target. The trails show the paths taken by seek and arrive. Arrive goes straight to its target, while seek orbits a bit and ends up oscillating. The oscillation is not as bad for dynamic seek as it was in kinematic seek: the character cannot change direction immediately, so it appears to wobble rather than shake around the target.

The dynamic arrive behavior is a little more complex than the kinematic version. It uses two radii. The arrival radius, as before, lets the character get near enough to the target without letting small errors keep it in motion. A second radius is also given, but is much larger. The incoming character will begin to slow down when it passes this radius. The algorithm calculates an ideal speed for the character. At the slowing-down radius, this is equal to its maximum speed. At the target point, it is zero (we want to have zero speed when we arrive). In between, the desired speed is an interpolated intermediate value, controlled by the distance from the target.

The direction toward the target is calculated as before. This is then combined with the desired speed to give a target velocity. The algorithm looks at the current velocity of the character and works out the acceleration needed to turn it into the target velocity. We can't immediately change velocity, however, so the acceleration is calculated based on reaching the target velocity in a fixed time scale.

This is exactly the same process as for kinematic arrive, where we tried to get the character

to arrive at its target in a quarter of a second. Because there is an extra layer of indirection (acceleration affects velocity which affects position) the fixed time period for dynamic arrive can usually be a little smaller; we'll use 0.1 as a good starting point.

When a character is moving too fast to arrive at the right time, its target velocity will be smaller than its actual velocity, so the acceleration is in the opposite direction—it is acting to slow the character down.

Pseudo-Code

The full algorithm looks like the following:

```

1  class Arrive:
2      character: Kinematic
3      target: Kinematic
4
5      maxAcceleration: float
6      maxSpeed: float
7
8      # The radius for arriving at the target.
9      targetRadius: float
10
11     # The radius for beginning to slow down.
12     slowRadius: float
13
14     # The time over which to achieve target speed.
15     timeToTarget: float = 0.1
16
17     function getSteering() -> SteeringOutput:
18         result = new SteeringOutput()
19
20         # Get the direction to the target.
21         direction = target.position - character.position
22         distance = direction.length()
23
24         # Check if we are there, return no steering.
25         if distance < targetRadius:
26             return null
27
28         # If we are outside the slowRadius, then move at max speed.
29         if distance > slowRadius:
30             targetSpeed = maxSpeed
31         # Otherwise calculate a scaled speed.
32         else:
33             targetSpeed = maxSpeed * distance / slowRadius
34
35         # The target velocity combines speed and direction.

```

```

36     targetVelocity = direction
37     targetVelocity.normalize()
38     targetVelocity *= targetSpeed
39
40     # Acceleration tries to get to the target velocity.
41     result.linear = targetVelocity - character.velocity
42     result.linear /= timeToTarget
43
44     # Check if the acceleration is too fast.
45     if result.linear.length() > maxAcceleration:
46         result.linear.normalize()
47         result.linear *= maxAcceleration
48
49     result.angular = 0
50     return result

```

Performance

The algorithm is $O(1)$ in both time and memory, as before.

Implementation Notes

Many implementations do not use a target radius. Because the character will slow down to reach its target, there isn't the same likelihood of oscillation that we saw in kinematic arrive. Removing the target radius usually makes no noticeable difference. It can be significant, however, with low frame rates or where characters have high maximum speeds and low accelerations. In general, it is good practice to give a margin of error around any target, to avoid annoying instabilities.

Leave

Conceptually, the opposite behavior of arrive is leave. There is no point in implementing it, however. If we need to leave a target, we are unlikely to want to accelerate with minuscule (possibly zero) acceleration first and then build up. We are more likely to accelerate as fast as possible. So for practical purposes the opposite of arrive is flee.

3.3.5 ALIGN

Align tries to match the orientation of the character with that of the target. It pays no attention to the position or velocity of the character or target. Recall that orientation is not directly

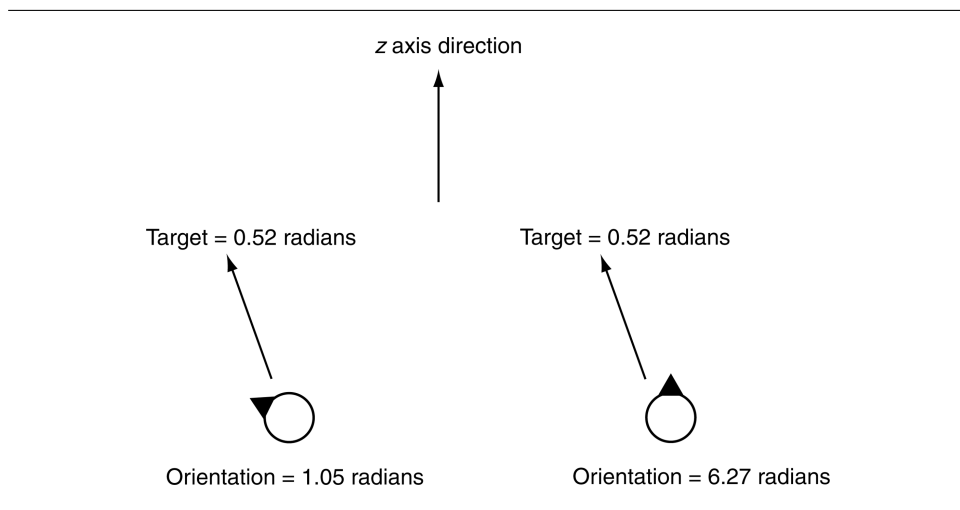


Figure 3.10: Aligning over a 2π radians boundary

related to direction of movement for a general kinematic. This steering behavior does not produce any linear acceleration; it only responds by turning.

Align behaves in a similar way to arrive. It tries to reach the target orientation and tries to have zero rotation when it gets there. Most of the code from arrive we can copy, but orientations have an added complexity that we need to consider.

Because orientations wrap around every 2π radians, we can't simply subtract the target orientation from the character orientation and determine what rotation we need from the result. Figure 3.10 shows two very similar align situations, where the character is the same angle away from its target. If we simply subtracted the two angles, the first one would correctly rotate a small amount clockwise, but the second one would travel all around to get to the same place.

To find the actual direction of rotation, we subtract the character orientation from the target and convert the result into the range $(-\pi, \pi)$ radians. We perform the conversion by adding or subtracting some multiple of 2π to bring the result into the given range. We can calculate the multiple to use by using the mod function and a little jiggling about. Most game engines or graphics libraries have one available (in Unreal Engine it is `FMath::FindDeltaAngle`, in Unity it is `Mathf.DeltaAngle`, though be aware that Unity uses degrees for its angles not radians).

We can then use the converted value to control rotation, and the algorithm looks very similar to arrive. Like arrive, we use two radii: one for slowing down and one to make orientations near the target acceptable. Because we are dealing with a single scalar value, rather than a 2D or 3D vector, the radius acts as an interval.

When we come to subtracting the rotation values, we have no problem we values repeat-

ing. Rotations, unlike orientations, don't wrap: a rotation of π is not the same as a rotational of 3π . In fact we can have huge rotation values, well out of the $(-\pi, \pi)$ range. Large values simply represent very fast rotation. Objects at very high speeds (such as the wheels in racing cars) may be rotating so fast that the updates we use here cause numeric instability. This is less of a problem on machines with 64-bit precision, but early physics on 32-bit machines often showed cars at high-speed having wheels that appeared to wobble. This has been resolved in robust physics engines, but it is worth knowing about, as the code we're discussing here is basically implementing a simple physics update. The simple approach is effective for moderate rotational speeds.

Pseudo-Code

Most of the algorithm is similar to arrive, we simply add the conversion:

```

1  class Align:
2      character: Kinematic
3      target: Kinematic
4
5      maxAngularAcceleration: float
6      maxRotation: float
7
8      # The radius for arriving at the target.
9      targetRadius: float
10
11     # The radius for beginning to slow down.
12     slowRadius: float
13
14     # The time over which to achieve target speed.
15     timeToTarget: float = 0.1
16
17     function getSteering() -> SteeringOutput:
18         result = new SteeringOutput()
19
20         # Get the naive direction to the target.
21         rotation = target.orientation - character.orientation
22
23         # Map the result to the (-pi, pi) interval.
24         rotation = mapToRange(rotation)
25         rotationSize = abs(rotation)
26
27         # Check if we are there, return no steering.
28         if rotationSize < targetRadius:
29             return null
30
31         # If we are outside the slowRadius, then use maximum rotation.
32         if rotationSize > slowRadius:

```

```

33         targetRotation = maxRotation
34     # Otherwise calculate a scaled rotation.
35     else:
36         targetRotation =
37             maxRotation * rotationSize / slowRadius
38
39     # The final target rotation combines speed (already in the
40     # variable) and direction.
41     targetRotation *= rotation / rotationSize
42
43     # Acceleration tries to get to the target rotation.
44     result.angular = targetRotation - character.rotation
45     result.angular /= timeToTarget
46
47     # Check if the acceleration is too great.
48     angularAcceleration = abs(result.angular)
49     if angularAcceleration > maxAngularAcceleration:
50         result.angular /= angularAcceleration
51         result.angular *= maxAngularAcceleration
52
53     result.linear = 0
54     return result

```

where the function `abs` returns the absolute (i.e., positive) value of a number; for example, `-1` is mapped to `1`.

Implementation Notes

Whereas in the arrive implementation there are two vector normalizations, in this code we need to normalize a scalar (i.e., turn it into either `+1` or `-1`). To do this we use the result that:

```
normalizedValue = value / abs(value)
```

In a production implementation in a language where you can access the bit pattern of a floating point number (C and C++, for example), you can do the same thing by manipulating the non-sign bits of the variable. Some C libraries provide an optimized `sign` function faster than the approach above.

Performance

The algorithm, unsurprisingly, is $O(1)$ in both memory and time.

The Opposite

There is no such thing as the opposite of align. Because orientations wrap around every 2π , fleeing from an orientation in one direction will simply lead you back to where you started. To face the opposite direction of a target, simply add π to its orientation and align to that value.

3.3.6 VELOCITY MATCHING

So far we have looked at behaviors that try to match position (linear position or orientation) with a target. We will do the same with velocity.

On its own this behavior is seldom used. It could be used to make a character mimic the motion of a target, but this isn't very useful. Where it does become critical is when combined with other behaviors. It is one of the constituents of the flocking steering behavior, for example.

We have already implemented an algorithm that tries to match a velocity. Arrive calculates a target velocity based on the distance to its target. It then tries to achieve the target velocity. We can strip the arrive behavior down to provide a velocity matching implementation.

Pseudo-Code

The stripped down code looks like the following:

```

1  class VelocityMatch:
2      character: Kinematic
3      target: Kinematic
4
5      maxAcceleration: float
6
7      # The time over which to achieve target speed.
8      timeToTarget = 0.1
9
10     function getSteering() -> SteeringOutput:
11         result = new SteeringOutput()
12
13         # Acceleration tries to get to the target velocity.
14         result.linear = target.velocity - character.velocity
15         result.linear /= timeToTarget
16
17         # Check if the acceleration is too fast.
18         if result.linear.length() > maxAcceleration:
19             result.linear.normalize()
20             result.linear *= maxAcceleration
21
22         result.angular = 0
23         return result

```

Performance

The algorithm is $O(1)$ in both time and memory.

3.3.7 DELEGATED BEHAVIORS

We have covered the basic building block behaviors that help to create many others. Seek and flee, arrive, and align perform the steering calculations for many other behaviors.

All the behaviors that follow have the same basic structure: they calculate a target, either a position or orientation (they could use velocity, but none of those we're going to cover does), and then they delegate to one of the other behaviors to calculate the steering. The target calculation can be based on many inputs. Pursue, for example, calculates a target for seek based on the motion of another target. Collision avoidance creates a target for flee based on the proximity of an obstacle. And wander creates its own target that meanders around as it moves.

In fact, it turns out that seek, align, and velocity matching are the only fundamental behaviors (there is a rotation matching behavior, by analogy, but I've never seen an application for it). As we saw in the previous algorithm, arrive can be divided into the creation of a (velocity) target and the application of the velocity matching algorithm. This is common. Many of the delegated behaviors below can, in turn, be used as the basis of another delegated behavior. Arrive can be used as the basis of pursue, pursue can be used as the basis of other algorithms, and so on.

In the code that follows I will use a polymorphic style of programming to capture these dependencies. You could alternatively use delegation, having the primitive algorithms called by the new techniques. Both approaches have their problems. In our case, when one behavior extends another, it normally does so by calculating an alternative target. Using inheritance means we need to be able to change the target that the super-class works on.

If we use the delegation approach, we'd need to make sure that each delegated behavior has the correct character data, `maxAcceleration`, and other parameters. This requires duplication and data copying that using sub-classes removes.

3.3.8 PURSUE AND EVADE

So far we have moved based solely on position. If we are chasing a moving target, then constantly moving toward its current position will not be sufficient. By the time we reach where it is now, it will have moved. This isn't too much of a problem when the target is close and we are reconsidering its location every frame. We'll get there eventually. But if the character is a long distance from its target, it will set off in a visibly wrong direction, as shown in [Figure 3.11](#).

Instead of aiming at its current position, we need to predict where it will be at some time in the future and aim toward that point. We did this naturally playing tag as children, which

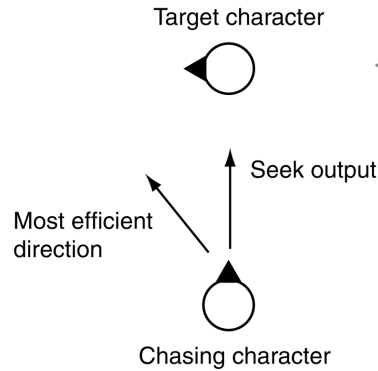


Figure 3.11: Seek moving in the wrong direction

is why the most difficult tag players to catch were those who kept switching direction, foiling our predictions.

We could use all kinds of algorithms to perform the prediction, but most would be overkill. Various research has been done into optimal prediction and optimal strategies for the character being chased (it is an active topic in military research for evading incoming missiles, for example). Craig Reynolds's original approach is much simpler: we assume the target will continue moving with the same velocity it currently has. This is a reasonable assumption over short distances, and even over longer distances it doesn't appear too stupid.

The algorithm works out the distance between character and target and works out how long it would take to get there, at maximum speed. It uses this time interval as its prediction lookahead. It calculates the position of the target if it continues to move with its current velocity. This new position is then used as the target of a standard seek behavior.

If the character is moving slowly, or the target is a long way away, the prediction time could be very large. The target is less likely to follow the same path forever, so we'd like to set a limit on how far ahead we aim. The algorithm has a maximum time parameter for this reason. If the prediction time is beyond this, then the maximum time is used.

Figure 3.12 shows a seek behavior and a pursue behavior chasing the same target. The pursue behavior is more effective in its pursuit.

Pseudo-Code

The pursue behavior derives from seek, calculates a surrogate target, and then delegates to seek to perform the steering calculation:

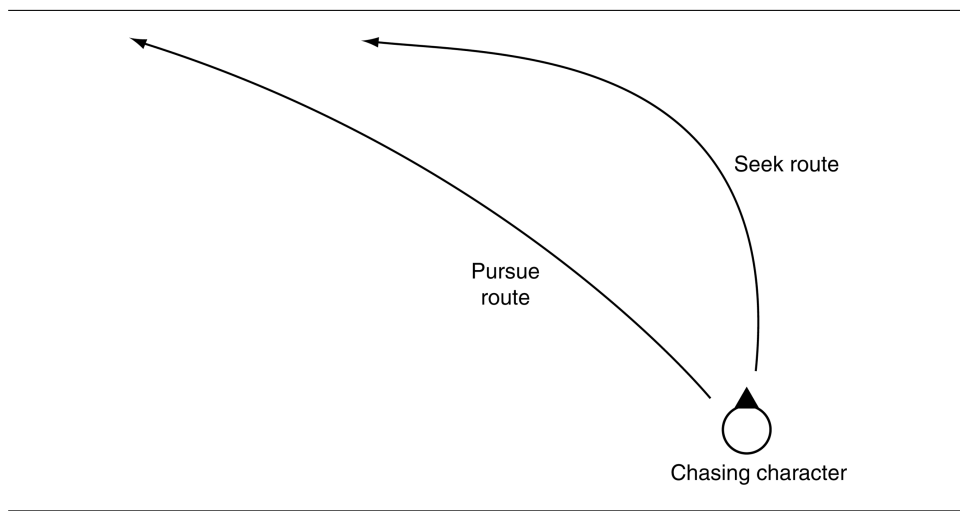


Figure 3.12: Seek and pursue

```

1  class Pursue extends Seek:
2      # The maximum prediction time.
3      maxPrediction: float
4
5      # OVERRIDES the target data in seek (in other words this class has
6      # two bits of data called target: Seek.target is the superclass
7      # target which will be automatically calculated and shouldn't be
8      # set, and Pursue.target is the target we're pursuing).
9      target: Kinematic
10
11     # ... Other data is derived from the superclass ...
12
13     function getSteering() -> SteeringOutput:
14         # 1. Calculate the target to delegate to seek
15         # Work out the distance to target.
16         direction = target.position - character.position
17         distance = direction.length()
18
19         # Work out our current speed.
20         speed = character.velocity.length()
21
22         # Check if speed gives a reasonable prediction time.
23         if speed <= distance / maxPrediction:
24             prediction = maxPrediction
25

```

```

26         # Otherwise calculate the prediction time.
27     else:
28         prediction = distance / speed
29
30     # Put the target together.
31     Seek.target = explicitTarget
32     Seek.target.position += target.velocity * prediction
33
34     # 2. Delegate to seek.
35     return Seek.getSteering()

```

Implementation Notes

In this code I've used the slightly unsavory technique of naming a member variable in a derived class with the same name as the super-class. In most languages this will have the desired effect of creating two members with the same name. In our case this is what we want: setting the pursue behavior's target will not change the target for the seek behavior it extends.

Be careful, though! In some languages (Python, for example) you can't do this. You'll have to name the target variable in each class with a different name.

As mentioned previously, it may be beneficial to cut out these polymorphic calls altogether to improve the performance of the algorithm. We can do this by having all the data we need in the pursue class, removing its inheritance of seek, and making sure that all the code the class needs is contained in the `getSteering` method. This is faster, but at the cost of duplicating the delegated code in each behavior that needs it and obscuring the natural reuse of the algorithm.

Performance

Once again, the algorithm is $O(1)$ in both memory and time.

Evade

The opposite behavior of pursue is evade. Once again we calculate the predicted position of the target, but rather than delegating to seek, we delegate to flee.

In the code above, the class definition is modified so that it is a sub-class of `Flee` rather than `Seek` and thus `Seek.getSteering` is changed to `Flee.getSteering`.

Overshooting

If the chasing character is able to move faster than the target, it will overshoot and oscillate around its target, exactly as the normal seek behavior does.

To avoid this, we can replace the delegated call to seek with a call to arrive. This illustrates the power of building up behaviors from their logical components; when we need a slightly different effect, we can easily modify the code to get it.

3.3.9 FACE

The face behavior makes a character look at its target. It delegates to the align behavior to perform the rotation but calculates the target orientation first.

The target orientation is generated from the relative position of the target to the character. It is the same process we used in the `getOrientation` function for kinematic movement.

Pseudo-Code

The implementation is very simple:

```

1  class Face extends Align:
2      # Overrides the Align.target member.
3      target: Kinematic
4
5      # ... Other data is derived from the superclass ...
6
7      # Implemented as it was in Pursue.
8      function getSteering() -> SteeringOutput:
9          # 1. Calculate the target to delegate to align
10         # Work out the direction to target.
11         direction = target.position - character.position
12
13         # Check for a zero direction, and make no change if so.
14         if direction.length() == 0:
15             return target
16
17         # 2. Delegate to align.
18         Align.target = explicitTarget
19         Align.target.orientation = atan2(-direction.x, direction.z)
20         return Align.getSteering()
```

3.3.10 LOOKING WHERE YOU'RE GOING

So far, I have assumed that the direction a character is facing does not have to be its direction of motion. In many cases, however, we would like the character to face in the direction it is moving. In the kinematic movement algorithms we set it directly. Using the align behavior, we can give the character angular acceleration to make it face the correct direction. In this way the character changes facing gradually, which can look more natural, especially for aerial

vehicles such as helicopters or hovercraft or for human characters that can move sideways (providing sidestep animations are available).

This is a process similar to the face behavior, above. The target orientation is calculated using the current velocity of the character. If there is no velocity, then the target orientation is set to the current orientation. We have no preference in this situation for any orientation.

Pseudo-Code

The implementation is even simpler than face:

```

1  class LookWhereYoureGoing extends Align:
2      # No need for an overridden target member, we have
3      # no explicit target to set.
4
5      # ... Other data is derived from the superclass ...
6
7      function getSteering() -> SteeringOutput:
8          # 1. Calculate the target to delegate to align
9          # Check for a zero direction, and make no change if so.
10         velocity: Vector = character.velocity
11         if velocity.length() == 0:
12             return null
13
14         # Otherwise set the target based on the velocity.
15         target.orientation = atan2(-velocity.x, velocity.z)
16
17         # 2. Delegate to align.
18         return Align.getSteering()
```

Implementation Notes

In this case we don't need another target member variable. There is no overall target; we are creating the current target from scratch. So, we can simply use `Align.target` for the calculated target (in the same way we did with pursue and the other derived algorithms).

Performance

The algorithm is $O(1)$ in both memory and time.

3.3.11 WANDER

The wander behavior controls a character moving aimlessly about.

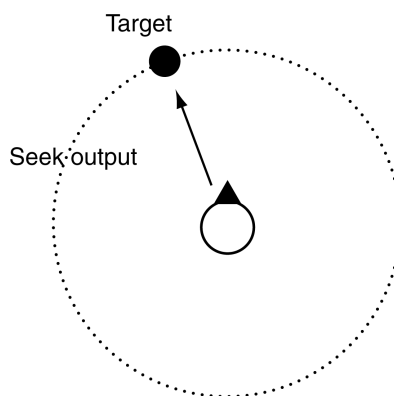


Figure 3.13: The kinematic wander as a seek

When we looked at the kinematic wander behavior, we perturbed the wander direction by a random amount each time it was run. This makes the character move forward smoothly, but the rotation of the character is erratic, appearing to twitch from side to side as it moves.

The simplest wander approach would be to move in a random direction. This is unacceptable, and therefore almost never used, because it produces linear jerkiness. The kinematic version added a layer of indirection but produced rotational jerkiness. We can smooth this twitching by adding an extra layer, making the orientation of the character indirectly reliant on the random number generator.

We can think of kinematic wander as behaving as a delegated seek behavior. There is a circle around the character on which the target is constrained. Each time the behavior is run, we move the target around the circle a little, by a random amount. The character then seeks the target. [Figure 3.13](#) illustrates this configuration.

We can improve this by moving the circle around which the target is constrained. If we move it out in front of the character (where front is determined by its current facing direction) and shrink it down, we get the situation in [Figure 3.14](#).

The character tries to face the target in each frame, using the face behavior to align to the target. It then adds an extra step: applying full acceleration in the direction of its current orientation.

We could also implement the behavior by having it seek the target and perform a ‘look where you’re going’ behavior to correct its orientation.

In either case, the orientation of the character is retained between calls (so smoothing the changes in orientation). The angles that the edges of the circle subtend to the character determine how fast it will turn. If the target is on one of these extreme points, it will turn quickly. The target will twitch and jitter around the edge of the circle, but the character’s orientation will change smoothly.

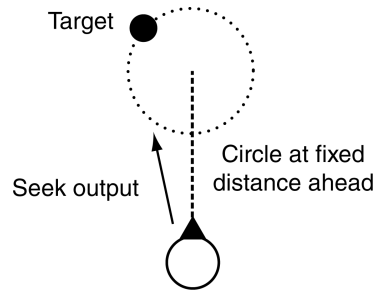


Figure 3.14: The full wander behavior

This wander behavior biases the character to turn (in either direction). The target will spend more time toward the edges of the circle, from the point of view of the character.

Pseudo-Code

```

1  class Wander extends Face:
2      # The radius and forward offset of the wander circle.
3      wanderOffset: float
4      wanderRadius: float
5
6      # The maximum rate at which the wander orientation can change.
7      wanderRate: float
8
9      # The current orientation of the wander target.
10     wanderOrientation: float
11
12     # The maximum acceleration of the character.
13     maxAcceleration: float
14
15     # Again we don't need a new target.
16     # ... Other data is derived from the superclass ...
17
18     function getSteering() -> SteeringOutput:
19         # 1. Calculate the target to delegate to face
20         # Update the wander orientation.
21         wanderOrientation += randomBinomial() * wanderRate
22
23         # Calculate the combined target orientation.
24         targetOrientation = wanderOrientation + character.orientation
25

```

```

26         # Calculate the center of the wander circle.
27         target = character.position +
28             wanderOffset * character.orientation.asVector()
29
30         # Calculate the target location.
31         target += wanderRadius * targetOrientation.asVector()
32
33         # 2. Delegate to face.
34         result = Face.getSteering()
35
36         # 3. Now set the linear acceleration to be at full
37         # acceleration in the direction of the orientation.
38         result.linear =
39             maxAcceleration * character.orientation.asVector()
40
41         # Return it.
42         return result

```

Data Structures and Interfaces

I've used the same `asVector` function as earlier to get a vector form of the orientation.

Performance

The algorithm is $O(1)$ in both memory and time.

3.3.12 PATH FOLLOWING

So far we've seen behaviors that take a single target or no target at all. Path following is a steering behavior that takes a whole path as a target. A character with path following behavior should move along the path in one direction.

Path following, as it is usually implemented, is a delegated behavior. It calculates the position of a target based on the current character location and the shape of the path. It then hands its target off to seek. There is no need to use arrive, because the target should always be moving along the path. We shouldn't need to worry about the character catching up with it.

The target position is calculated in two stages. First, the current character position is mapped to the nearest point along the path. This may be a complex process, especially if the path is curved or made up of many line segments. Second, a target is selected which is further along the path than the mapped point by some distance. To change the direction of motion along the path, we can change the sign of this distance. [Figure 3.15](#) shows this in action. The current path location is shown, along with the target point a little way farther along. This

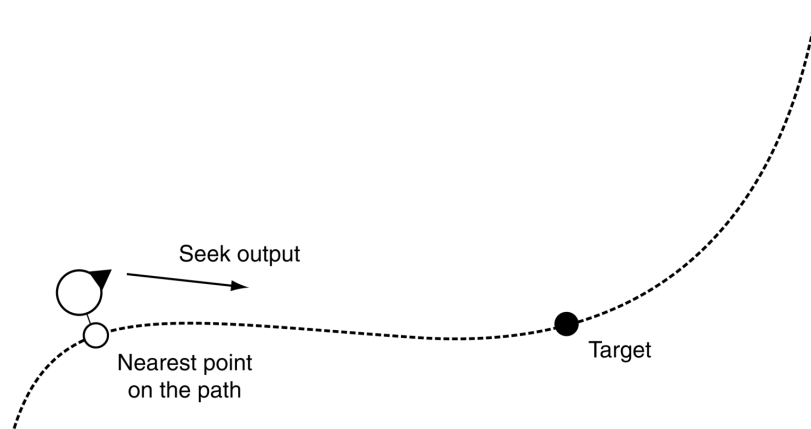


Figure 3.15: Path following behavior

approach is sometimes called “chase the rabbit,” after the way greyhounds chase the cloth rabbit at the dog track.

Some implementations generate the target slightly differently. They first predict where the character will be in a short time and then map this to the nearest point on the path. This is a candidate target. If the new candidate target has not been placed farther along the path than it was at the last frame, then it is changed so that it is. I will call this *predictive path following*. It is shown in [Figure 3.16](#). This latter implementation can appear smoother for complex paths with sudden changes of direction, but has the downside of cutting corners when two paths come close together.

[Figure 3.17](#) shows this cutting-corner behavior. The character misses a whole section of the path. The character is shown at the instant its predictive future position crosses to a later part of the path.

This might not be what you want if, for example, the path represents a patrol route or a race track.

Pseudo-Code

```

1 class FollowPath extends Seek:
2     path: Path
3
4     # The distance along the path to generate the target. Can be
5     # negative if the character is moving in the reverse direction.
6     pathOffset: float
7 
```

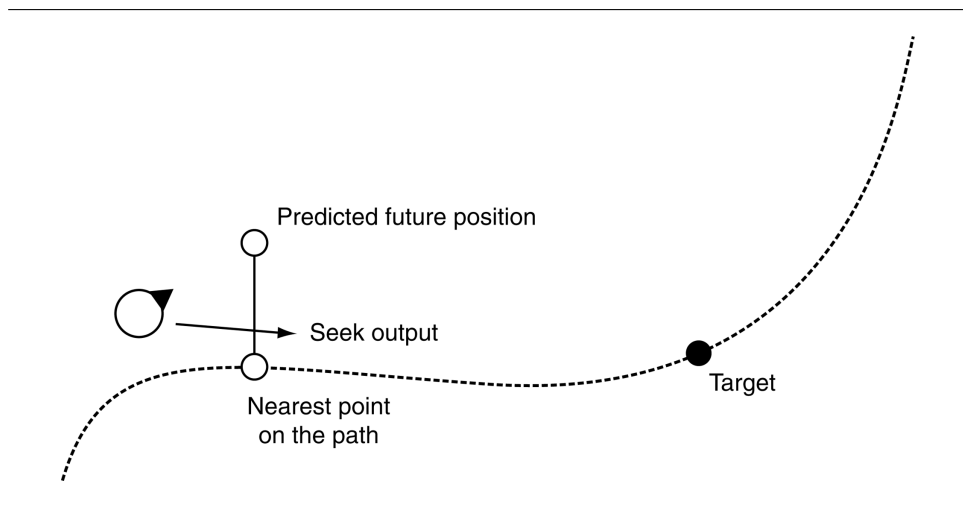


Figure 3.16: Predictive path following behavior

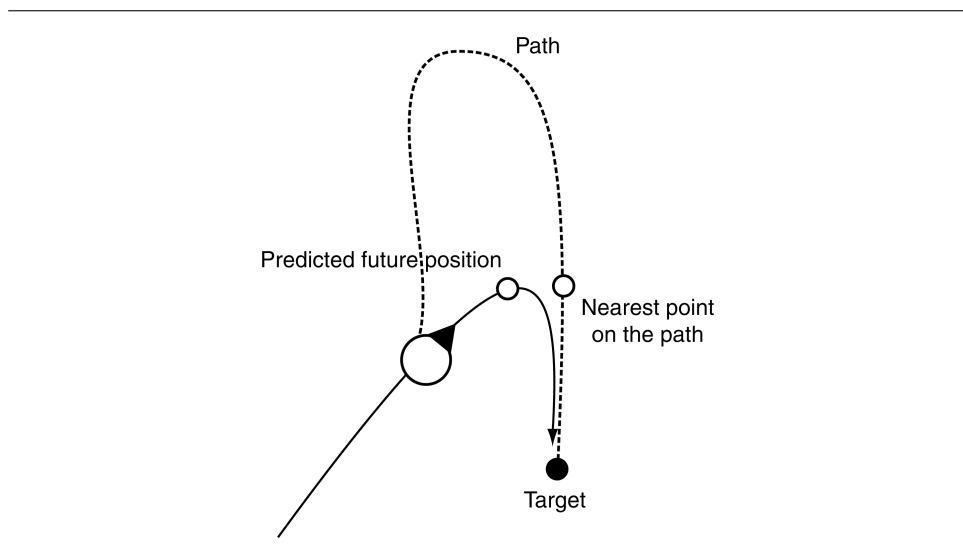


Figure 3.17: Vanilla and predictive path following

```

8      # The current position on the path.
9      currentParam: float
10
11     # ... Other data is derived from the superclass ...
12
13     function getSteering() -> SteeringOutput:
14         # 1. Calculate the target to delegate to face.
15         # Find the current position on the path.
16         currentParam = path.getParam(character.position, currentPos)
17
18         # Offset it.
19         targetParam = currentParam + pathOffset
20
21         # Get the target position.
22         target.position = path.getPosition(targetParam)
23
24         # 2. Delegate to seek.
25         return Seek.getSteering( )

```

We can convert this algorithm to a predictive version by first calculating a surrogate position for the call to `path.getParam`. The algorithm looks almost identical:

```

1  class FollowPath extends Seek:
2      path: Path
3
4      # The distance along the path to generate the target. Can be
5      # negative if the character is moving in the reverse direction.
6      pathOffset: float
7
8      # The current position on the path.
9      currentParam: float
10
11     # The time in the future to predict the character's position.
12     predictTime: float = 0.1
13
14     # ... Other data is derived from the superclass ...
15
16     function getSteering() -> SteeringOutput:
17         # 1. Calculate the target to delegate to face.
18         # Find the predicted future location.
19         futurePos = character.position +
20                     character.velocity * predictTime
21
22         # Find the current position on the path.
23         currentParam = path.getParam(futurePos, currentPos)
24
25         # Offset it.
26         targetParam = currentParam + pathOffset

```

```

27
28     # Get the target position.
29     target.position = path.getPosition(targetParam)
30
31     # 2. Delegate to seek.
32     return Seek.getSteering( )

```

Data Structures and Interfaces

The path that the behavior follows has the following interface:

```

1  class Path:
2      function getParam(position: Vector, lastParam: float) -> float
3      function getPosition(param: float) -> Vector

```

Both these functions use the concept of a path parameter. This is a unique value that increases monotonically along the path. It can be thought of as a distance along the path. Typically, paths are made up of line or curve splines; both of these are easily assigned parameters. The parameter allows us to translate between the position on the path and positions in 2D or 3D space.

Path Types

Performing this translation (i.e., implementing a path class) can be tricky, depending on the format of the path used.

It is most common to use a path of straight line segments as shown in [Figure 3.18](#). In this case the conversion is not too difficult. We can implement `getParam` by looking at each line segment in turn, determining which one the character is nearest to, and then finding the nearest point on that segment. For smooth curved splines common in some driving games, however, the math can be more complex. A good source for closest-point algorithms for a range of different geometries is Schneider and Eberly [56].

Keeping Track of the Parameter

The pseudo-code interface above provides for sending the last parameter value to the path in order to calculate the current parameter value. This is essential to avoid nasty problems when lines are close together.

We limit the `getParam` algorithm to only considering areas of the path close to the previous parameter value. The character is unlikely to have moved far, after all. This technique, assuming the new value is close to the old one, is called *coherence*, and it is a feature of many geometric algorithms. [Figure 3.19](#) shows a problem that would fox a non-coherent path follower but is easily handled by assuming the new parameter is close to the old one.

Of course, you may really want corners to be cut or a character to move between very different parts of the path. If another behavior interrupts and takes the character across the

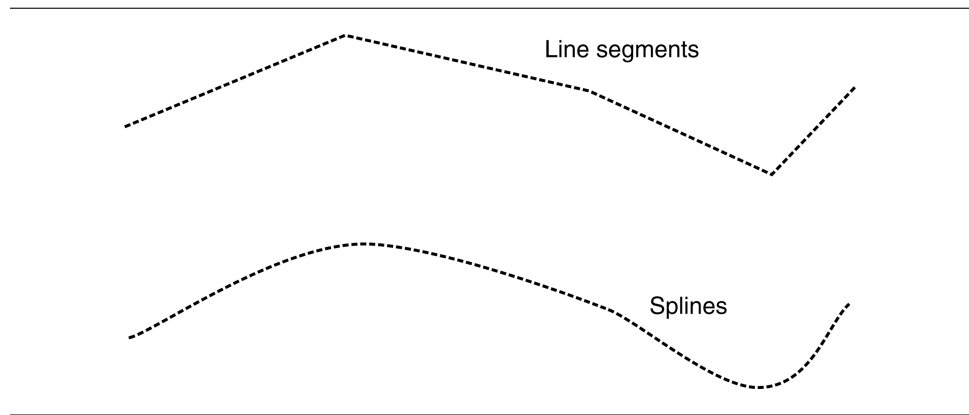


Figure 3.18: Path types

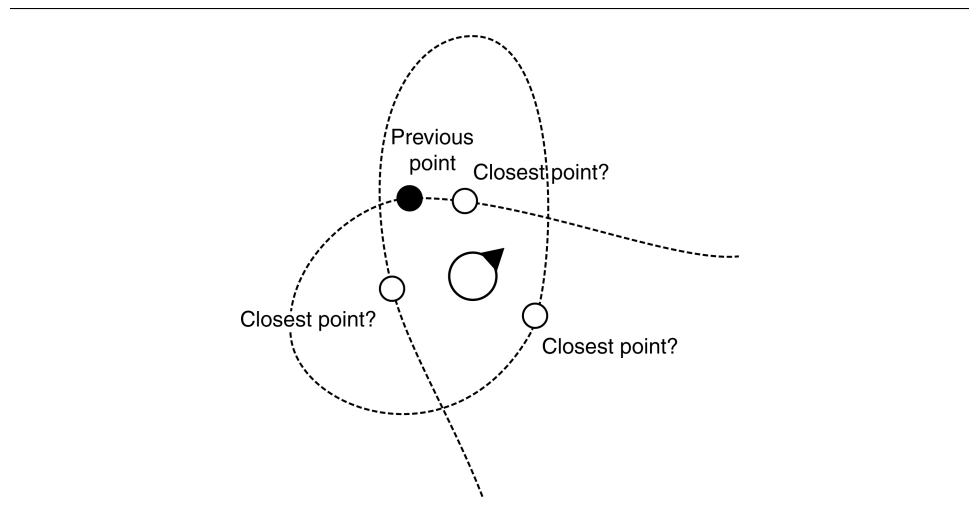


Figure 3.19: Coherence problems with path following

level, for example, you don't necessarily want it to come all the way back to pick up a circular patrol route. In this case, you'll need to remove coherence or at least widen the range of parameters that it searches for a solution.

Performance

The algorithm is $O(1)$ in both memory and time. The `getParam` function of the path will usually be $O(1)$, although it may be $O(n)$, where n is the number of segments in the path. If this is the case, then the `getParam` function will dominate the performance scaling of the algorithm.

3.3.13 SEPARATION

The separation behavior is common in crowd simulations, where a number of characters are all heading in roughly the same direction. It acts to keep the characters from getting too close and being crowded.

It doesn't work as well when characters are moving across each other's paths. The collision avoidance behavior, below, should be used in this case.

Most of the time, the separation behavior has a zero output; it doesn't recommend any movement at all. If the behavior detects another character closer than some threshold, it acts in a way similar to an evade behavior to move away from the character. Unlike the basic evade behavior, however, the strength of the movement is related to the distance from the target. The separation strength can decrease according to any formula, but a linear or an inverse square law decay is common.

Linear separation looks like the following:

```
strength = maxAcceleration * (threshold - distance) / threshold
```

The inverse square law looks like the following:

```
strength = min(k / (distance * distance), maxAcceleration)
```

In each case, `distance` is the distance between the character and its nearby neighbor, `threshold` is the minimum distance at which any separation output occurs, and `maxAcceleration` is the maximum acceleration of the character. The `k` constant can be set to any positive value. It controls how fast the separation strength decays with distance.

Separation is sometimes called the "repulsion" steering behavior, because it acts in the same way as a physical repulsive force (an inverse square law force such as magnetic repulsion).

Where there are multiple characters within the avoidance threshold, either the closest can be selected, or the steering can be calculated for each in turn and summed. In the latter case, the final value may be greater than the `maxAcceleration`, in which case it should be clipped to that value.

Pseudo-Code

```

1  class Separation:
2      character: Kinematic
3      maxAcceleration: float
4
5      # A list of potential targets.
6      targets: Kinematic[]
7
8      # The threshold to take action.
9      threshold: float
10
11     # The constant coefficient of decay for the inverse square law.
12     decayCoefficient: float
13
14     function getSteering() -> SteeringOutput:
15         result = new SteeringOutput()
16
17         # Loop through each target.
18         for target in targets:
19             # Check if the target is close.
20             direction = target.position - character.position
21             distance = direction.length()
22
23             if distance < threshold:
24                 # Calculate the strength of repulsion
25                 # (here using the inverse square law).
26                 strength = min(
27                     decayCoefficient / (distance * distance),
28                     maxAcceleration)
29
30                 # Add the acceleration.
31                 direction.normalize()
32                 result.linear += strength * direction
33
34         return result

```

Implementation Notes

In the algorithm above, we simply look at each possible character in turn and work out whether we need to separate from them. For a small number of characters, this will be the fastest approach. For a few hundred characters in a level, we need a faster method.

Typically, graphics and physics engines rely on techniques to determine what objects are close to one another. Objects are stored in spatial data structures, so it is relatively easy to